

Efficient Monte-Carlo event generation for neutrino-nucleus exclusive cross sections

Mathias El Baz, University of Geneva

NuWro workshop 2026
18/09/2026

I. Motivations

II. Efficient sampling with Normalizing Flows

III. NuGeNFlow and its interface to NEUT

IV. Bonus

Implementations in MC generators

- Most of the implemented models provide only inclusive cross sections.
- Exclusive implementations rely on factorization

$$\frac{d^6\sigma_{\nu\text{-Nucleus}}}{dE_\mu d\Omega_\mu d\Omega_N dE_N} \sim \underbrace{S(E_m, p_m)}_{\text{Spectral function}} \underbrace{L_{\mu\nu} W^{\mu\nu} \delta(\omega + M - E_m - E_p)}_{\text{Neutrino-nucleon Cross-section}}$$

⇒ This neglects final state interactions (elastic and inelastic)

Implementations in MC generators

- Most of the implemented models provide only inclusive cross sections.
- Exclusive implementations rely on factorization

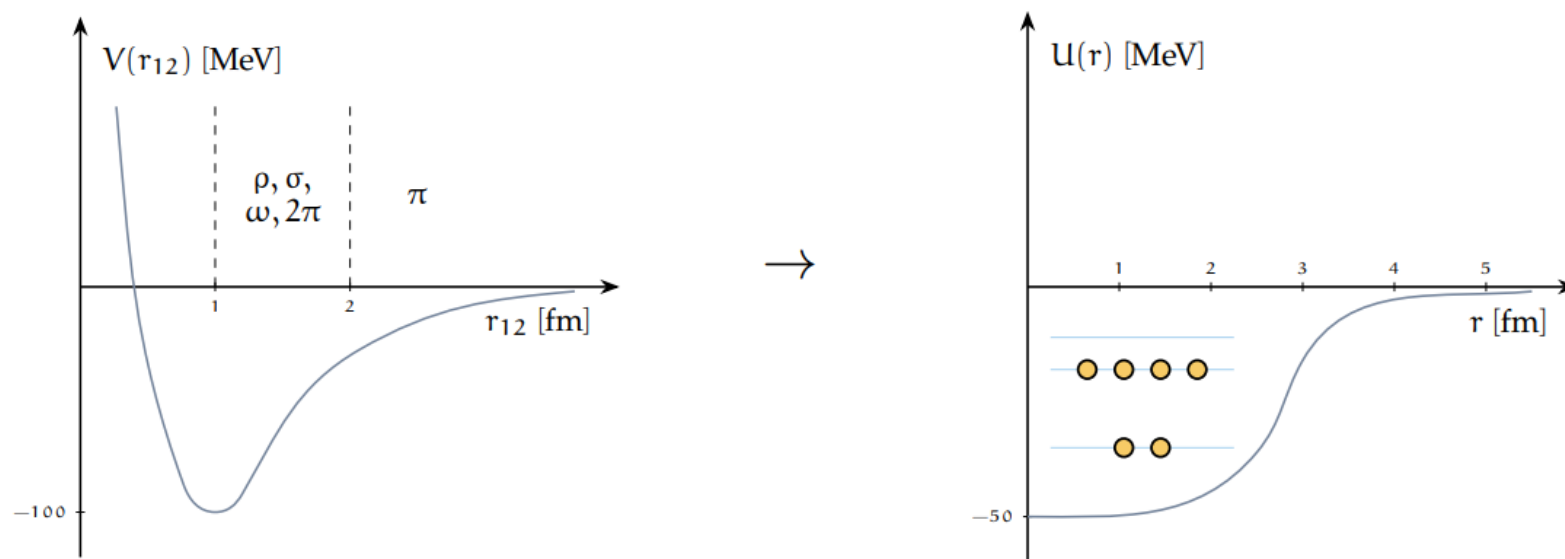
$$\frac{d^6\sigma_{\nu\text{-Nucleus}}}{dE_\mu d\Omega_\mu d\Omega_N dE_N} \sim \underbrace{S(E_m, p_m)}_{\text{Spectral function}} \underbrace{L_{\mu\nu} W^{\mu\nu} \delta(\omega + M - E_m - E_p)}_{\text{Neutrino-nucleon Cross-section}}$$

⇒ This neglects final state interactions (elastic and inelastic)

How to go beyond factorization ?
 Outgoing nucleon state = Sum of distorted wave solutions to a nuclear potential (EDRMF, ROP...) instead of a plane wave

Hartree-Fock Mean field

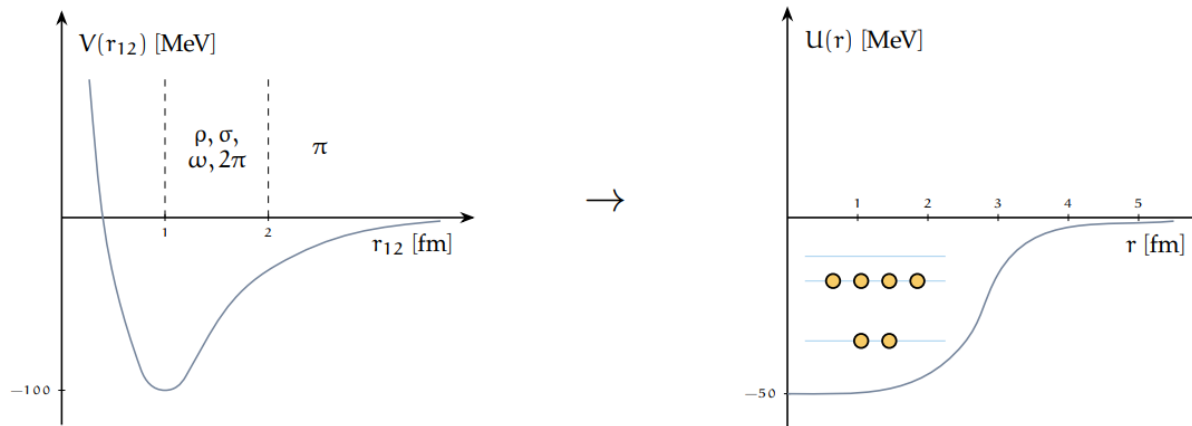
- **Shell** model describing nucleons in a **mean nucleus potential**.
- Nucleon-nucleus potential derived iteratively via a **Hartree-Fock** procedure from an effective **Skyrme** nucleon-nucleon potential.



*From Nucleon-nucleon potential to Nucleon-Nucleus potential
(Kajetan Niewczas)*

Hartree-Fock Mean field

- **Shell** model describing nucleons in a **mean nucleus potential**.
- Nucleon-nucleus potential derived iteratively via a **Hartree-Fock** procedure from an effective **Skyrme** nucleon-nucleon potential.

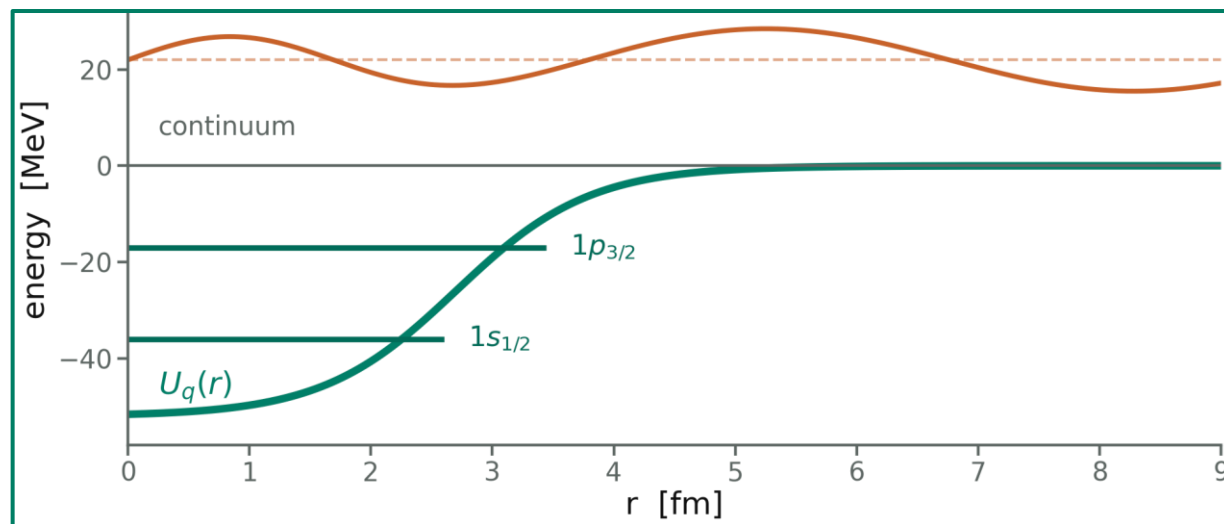


*From Nucleon-nucleon potential to Nucleon-Nucleus potential
(Kajetan Niewczas)*

- Both initial and final nucleon state **solutions of the Schrödinger equation** with the Mean Field potential

Hartree-Fock Mean field

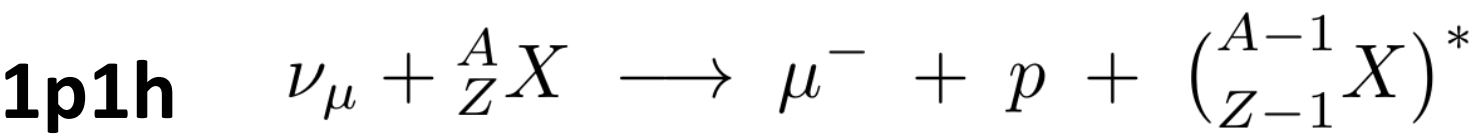
- **Shell** model describing nucleons in a **mean nucleus potential**.
- Nucleon-nucleus potential derived iteratively via a **Hartree-Fock** procedure from an effective **Skyrme** nucleon-nucleon potential.



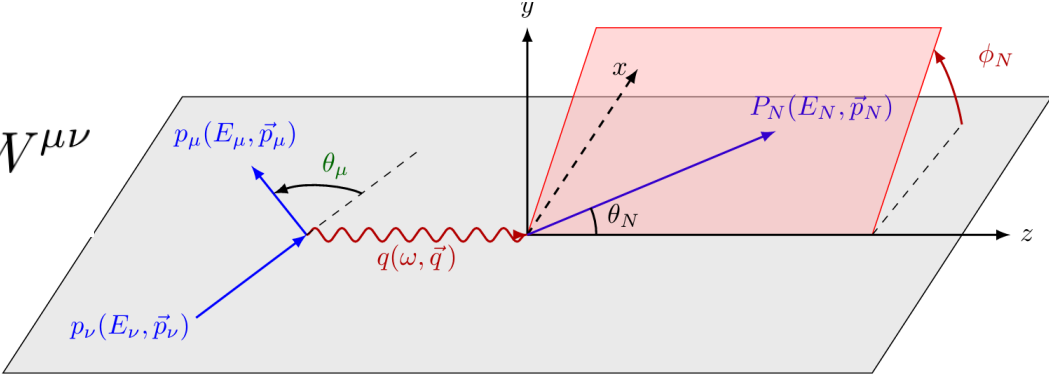
Solutions in the Mean Field potential

- Both initial and final nucleon state **solutions of the Schrödinger equation** with the Mean Field potential

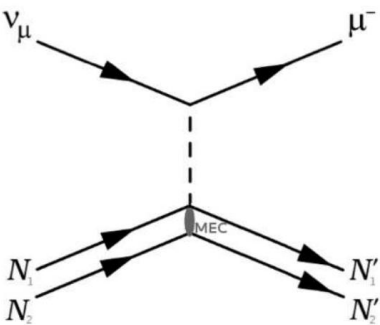
Cross sections predictions



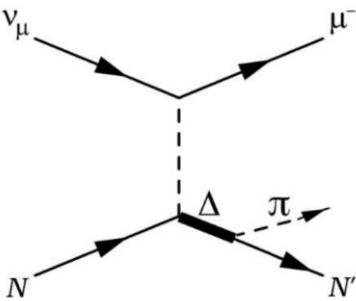
$$\frac{d^6\sigma(E_\nu)}{d\omega\,d\Omega_\mu\,dE_N\,d\Omega_N} \propto L_{\mu\nu}W^{\mu\nu}$$



2p2h



SPP



Ingredients for a slow sampling

Summation of a huge
number of solutions

Ingredients for a slow sampling

Summation of a huge
number of solutions



Time-consuming cross-
section computations

Ingredients for a slow sampling

Summation of a huge
number of solutions



Time-consuming cross-
section computations

Simple accept-reject
sampling method

Ingredients for a slow sampling

Summation of a huge
number of solutions



Time-consuming cross-
section computations

Simple accept-reject
sampling method



Inefficient sampling in
high dimension

Ingredients for a slow sampling

Summation of a huge number of solutions

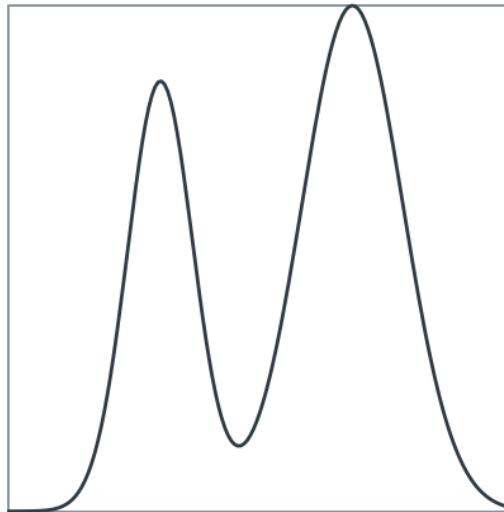


Time-consuming cross-section computations

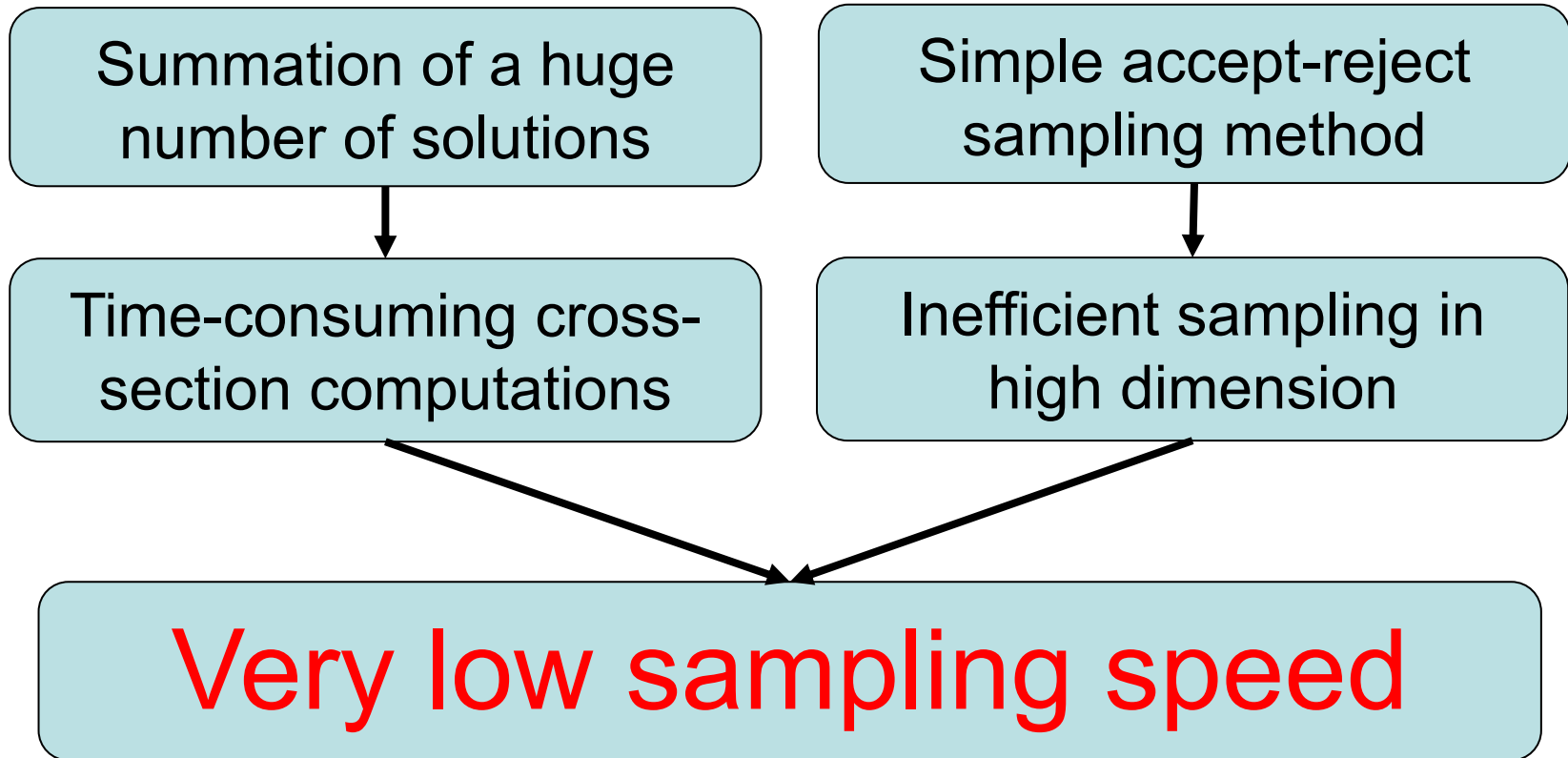
Simple accept-reject sampling method



Inefficient sampling in high dimension



Ingredients for a slow sampling



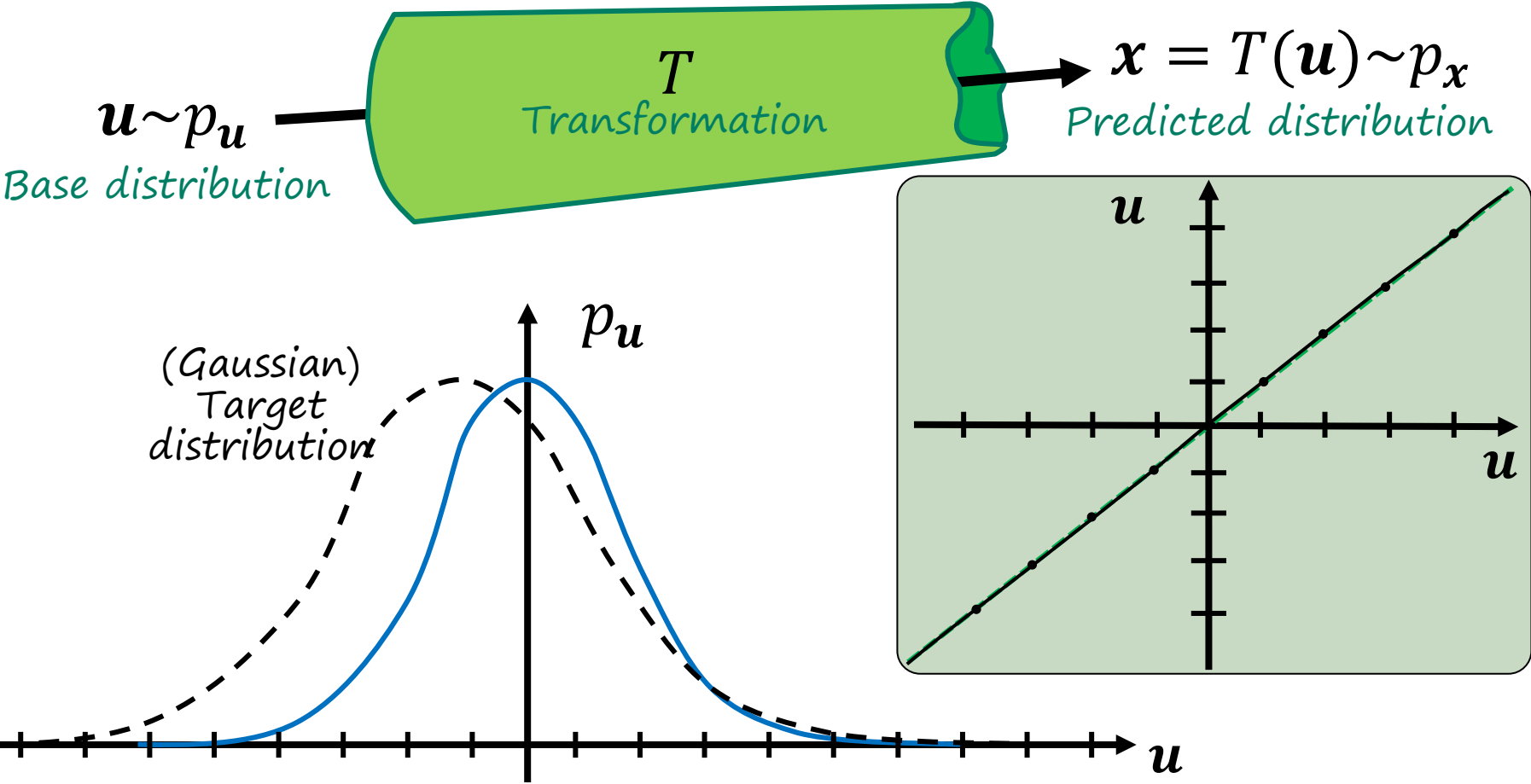
I. Motivations

II. Efficient sampling with
Normalizing Flows

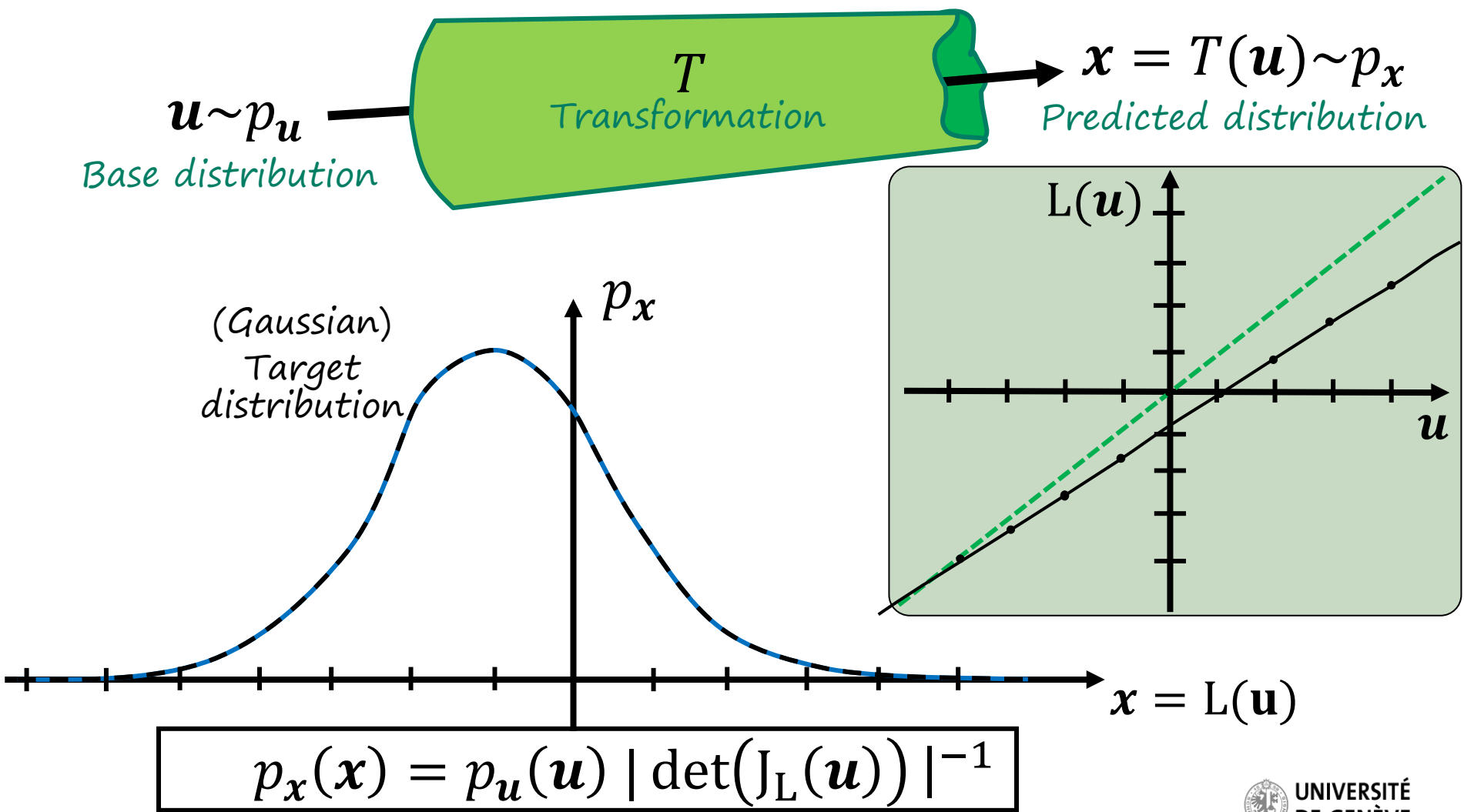
III. NuGeNFlow and
its interface to NEUT

IV. Bonus

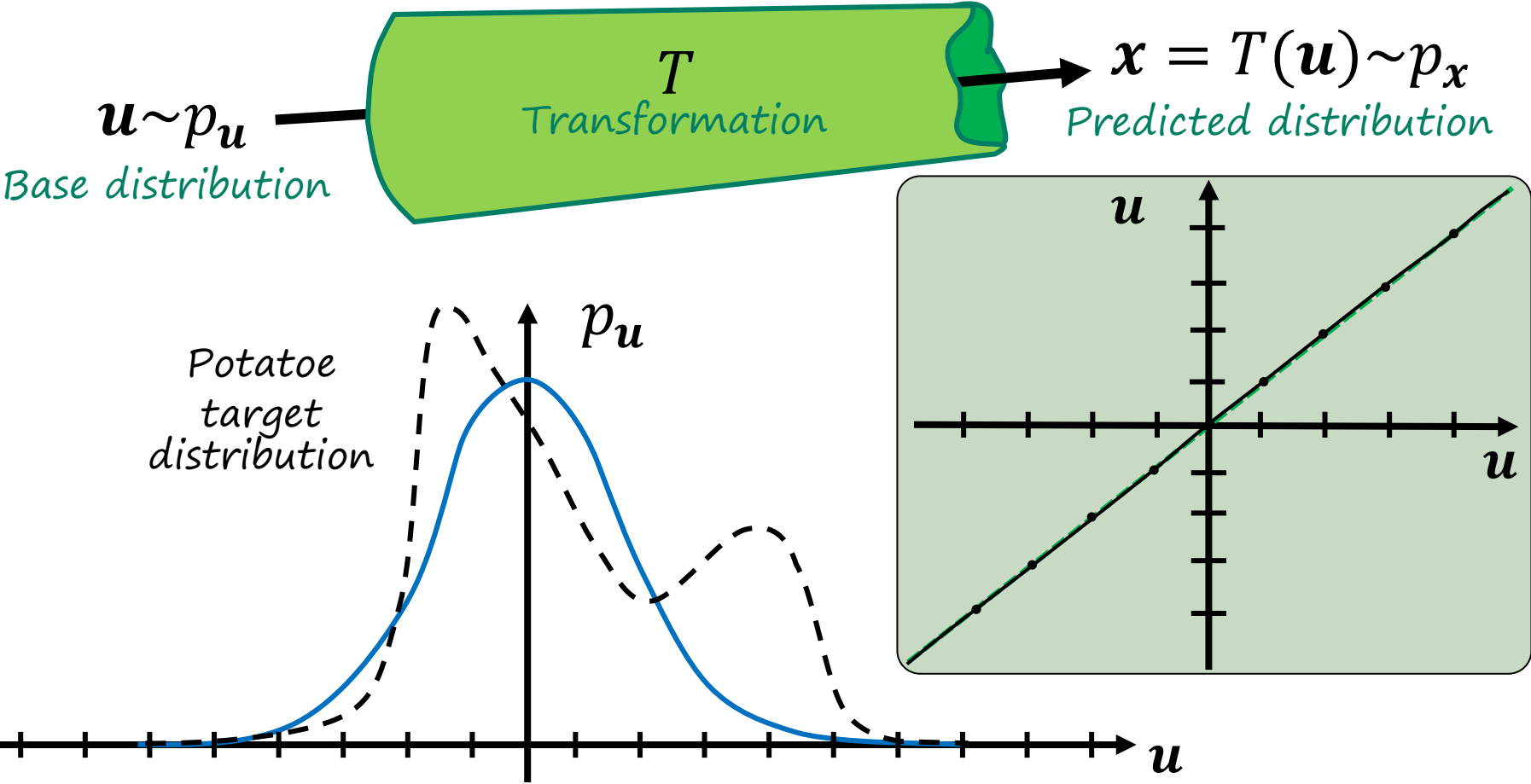
Normalizing flows



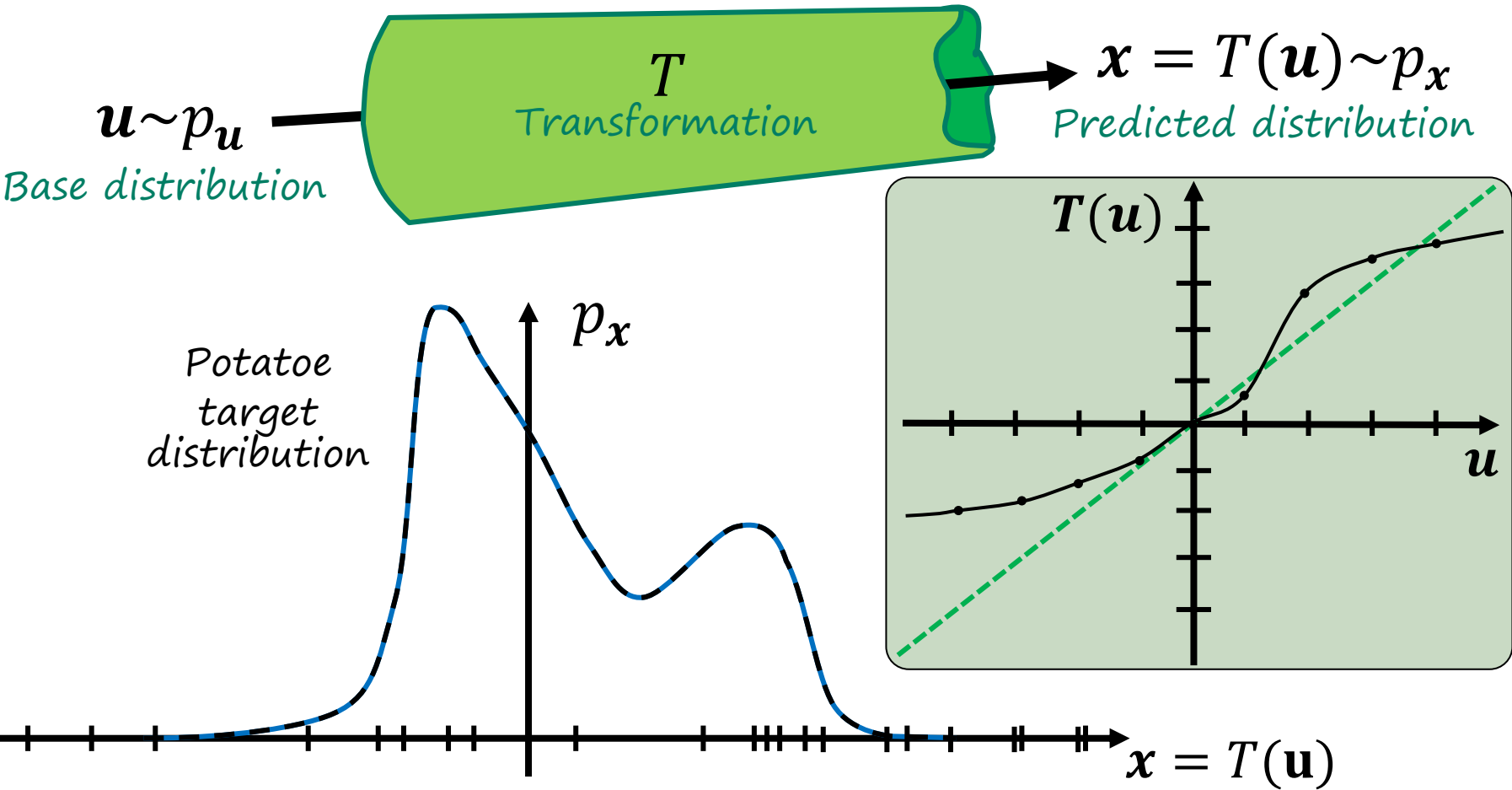
Normalizing flows



Normalizing flows



Normalizing flows



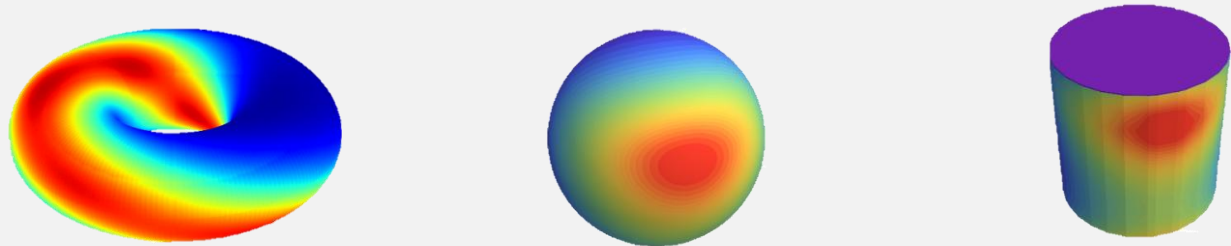
$$p_x(\mathbf{x}) = p_u(\mathbf{u}) \left| \det(\mathbf{J}_T(\mathbf{u})) \right|^{-1}$$

Normalizing flows

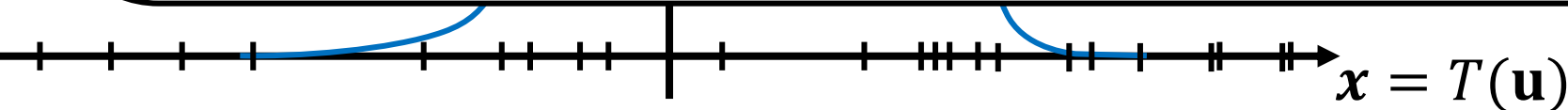


Can be extended to :

- Multidimensional probability space (autoregressive flows)
- Some non-euclidian manifolds (product of sphere, tori...)



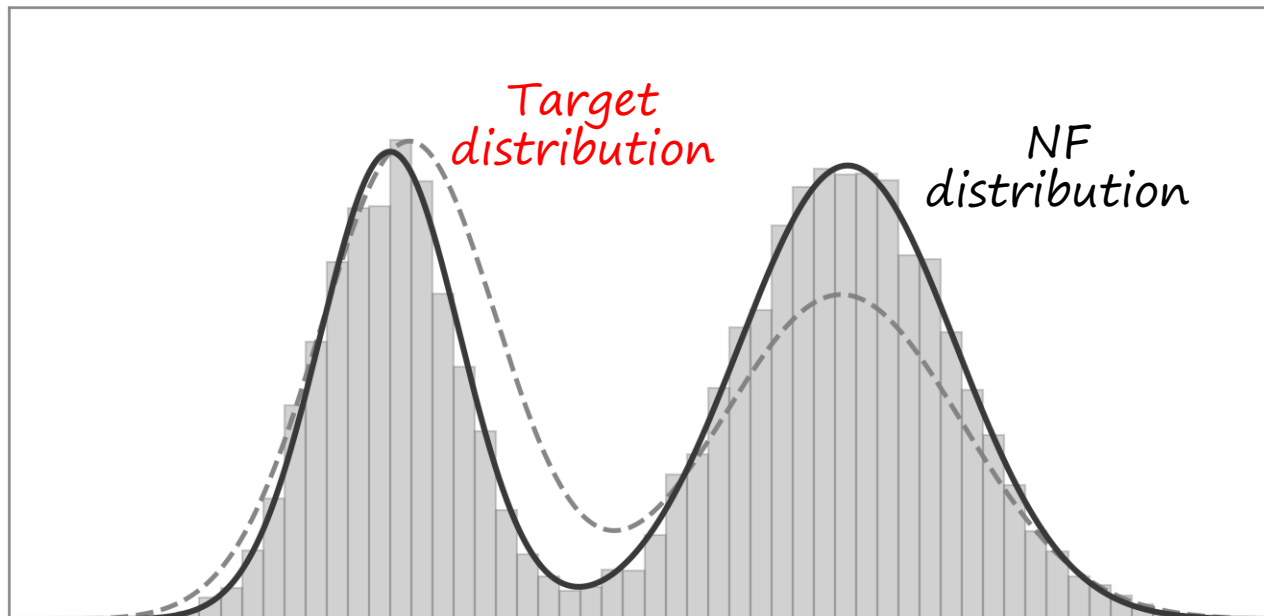
<https://arxiv.org/pdf/2002.02428>



$$p_x(x) = p_u(u) | \det(J_T(u)) |^{-1}$$

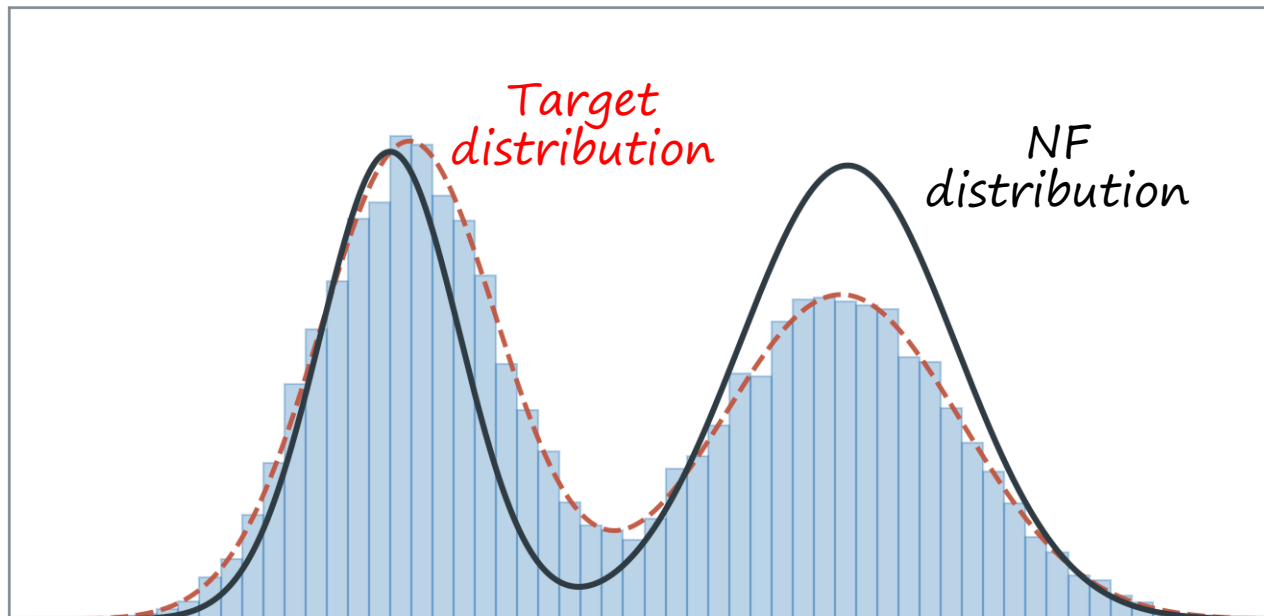
How to use the Normalizing Flows ?

As a proposal sampler for
accept-reject/importance
sampling



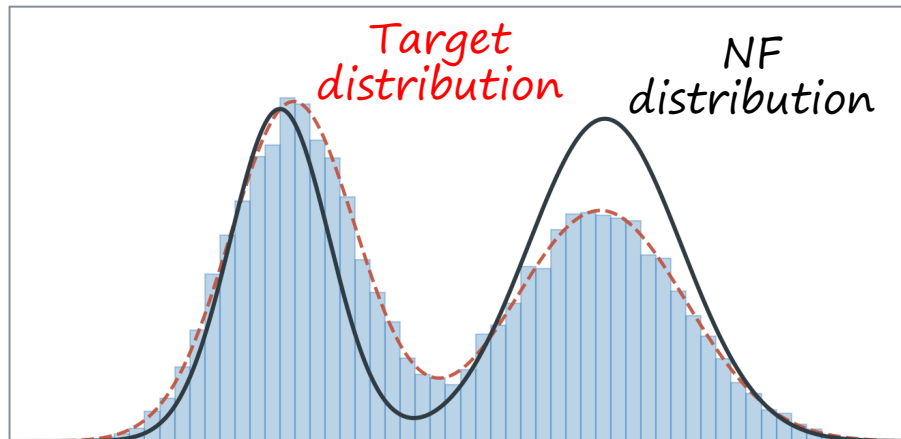
How to use the Normalizing Flows ?

As a proposal sampler for
accept-reject/importance
sampling



How to use the Normalizing Flows ?

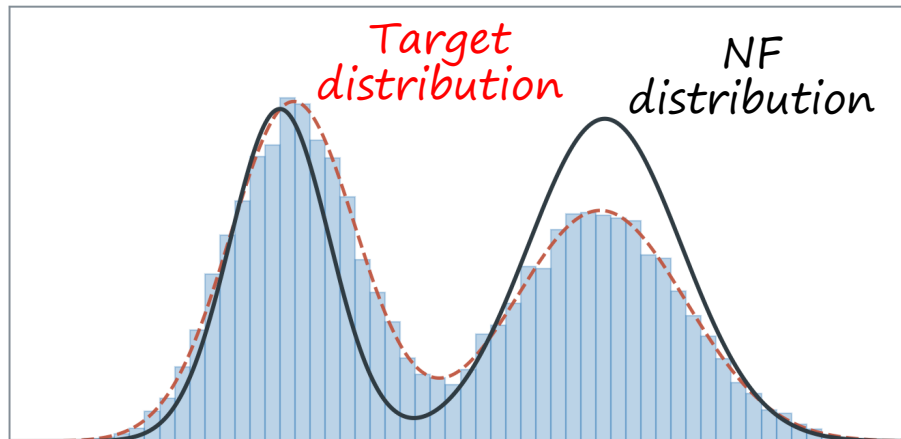
As a proposal sampler for
accept-reject/importance
sampling



How to use the Normalizing Flows ?

As a proposal sampler for
accept-reject/importance
sampling

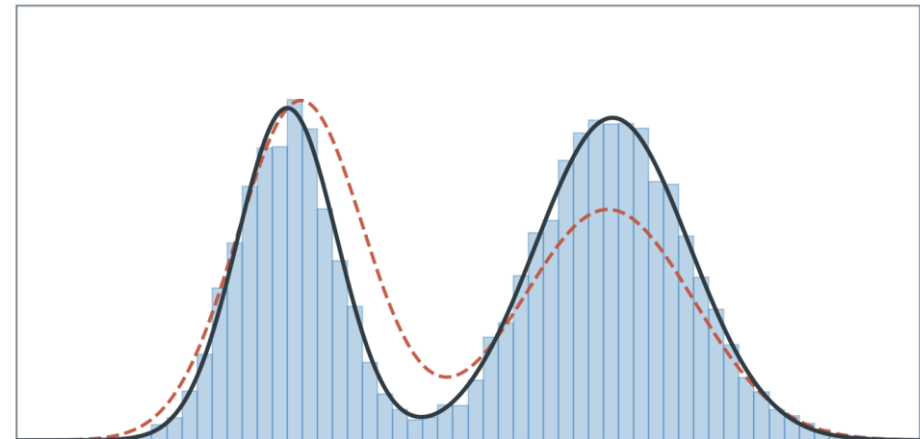
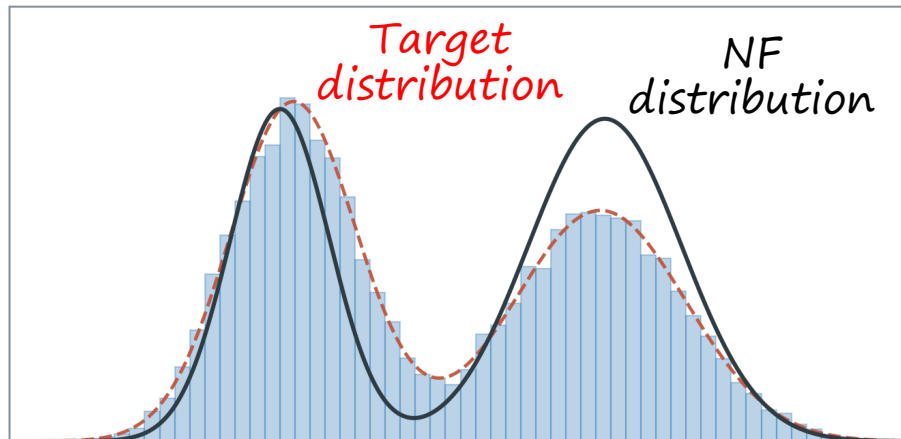
As a sample
emulator



How to use the Normalizing Flows ?

As a proposal sampler for
accept-reject/importance
sampling

As a sample
emulator



How to use the Normalizing Flows ?

As a proposal sampler for
accept-reject/importance
sampling

As a sample
emulator

Advantages and inconvenients

- + Asymptotically unbiased
- + Optimized acceptance rate
- Time consuming
- Xsec code required
- Non-differentiable (AR)

- + Extremely fast
- + Differentiable
- + Independent of xsec code
- Asymptotically biased

How to use the Normalizing Flows ?

As a proposal sampler for
accept-reject/importance
sampling

As a sample
emulator

How to evaluate the performances ?

At the distribution level

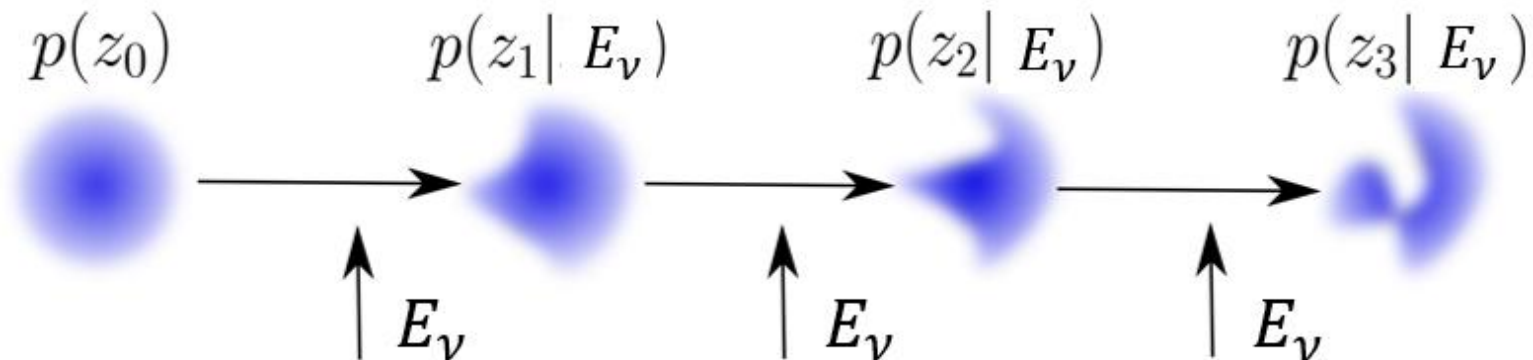
Effective sampling size
Acceptance rate ...

At the dataset level

Distance to an unbiased
dataset (χ^2 etc...)

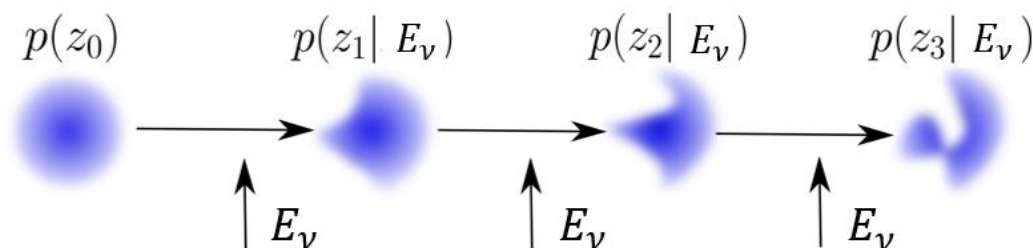
Energy Dependent Flows

- Neutrino event generators sample events based on the **conditional cross section given E_ν** .
- We mimic this by using **conditional NF** conditioned on E_ν .



Energy Dependent Flows

- Neutrino event generators sample events based on the **conditional cross section given E_ν** .
- We mimic this by using **conditional NF** conditioned on E_ν .



No need to retrain for different E_ν !

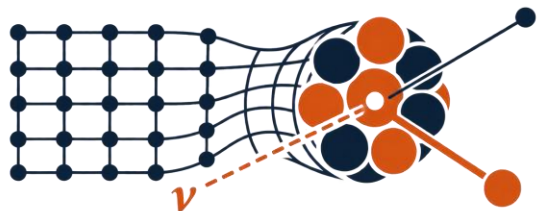
**And this can be extended to other conditional parameters
(spoiler for the end of the talk)**

I. Motivations

II. Efficient sampling with
Normalizing Flows

III. NuGeNFlow and
its interface to NEUT

IV. Bonus



Work in progress logo

 NuGenFlow Private

 Edit Pins  Watch

 master  2 Branches  0 Tags

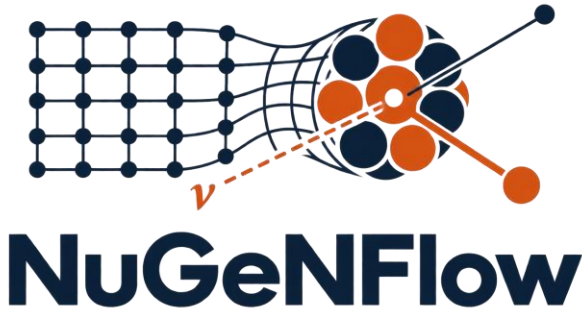
 Go to file 

 Add file 

 Code

 Mathiaselbaz Merge pull request #7 from NuMeanFielders/dev/Hal8180 	5ab78fe · yesterday	 41 Commits
 bash	Training for toy and gfg works, added some debugging plo...	2 weeks ago
 configs	Improved diagnostic plots	2 weeks ago
 docs	Corrected apptainer.def and added .md doc about it	yesterday
 env	Adding Apptainer.def for HPC usages	yesterday
 src	Removed useless dummy function	last week

This will change soon



Work in progress logo

master

2 Branches

0 Tags

Go to file

Add file

Code

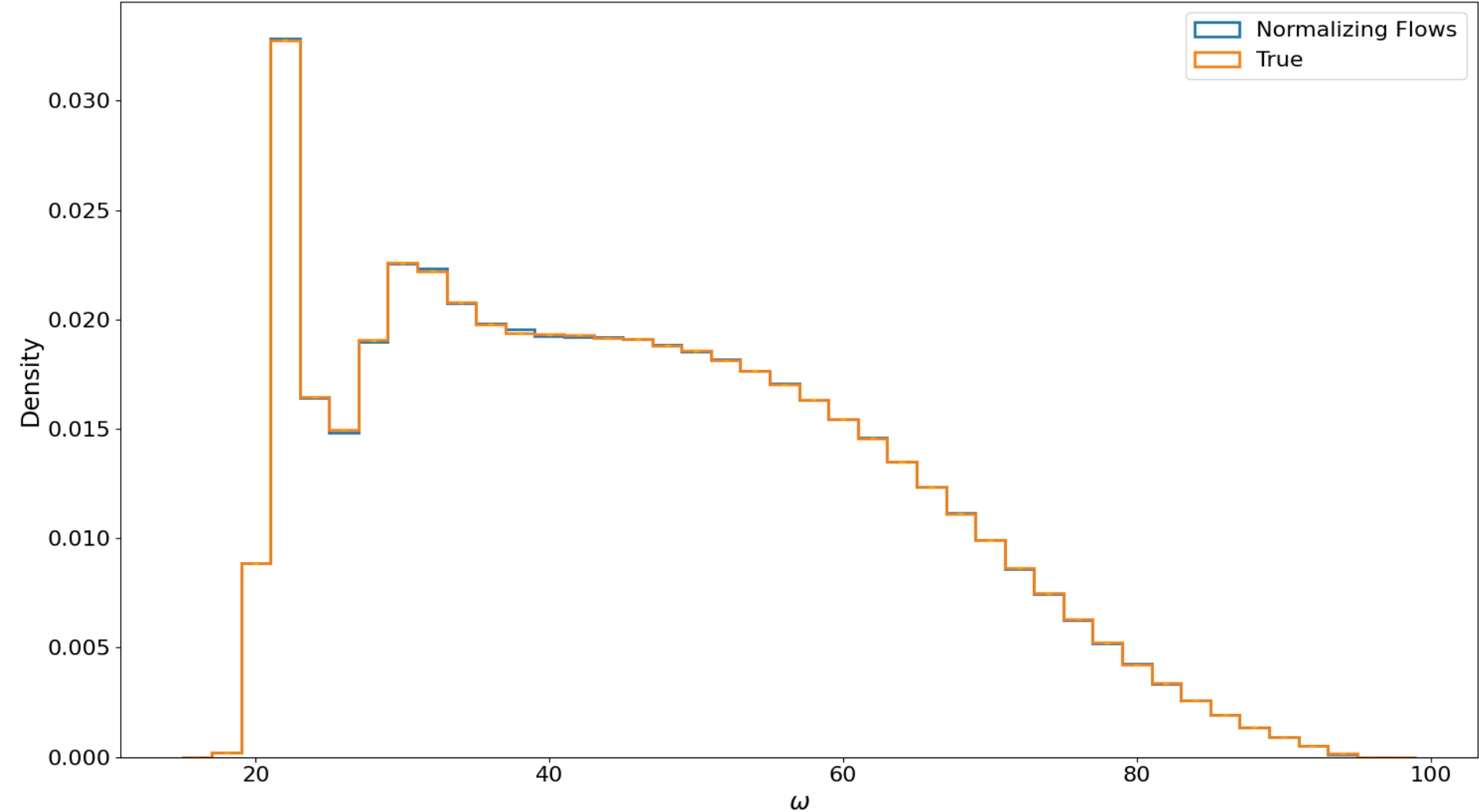
Mathiaselbaz Merge pull request [#7](#) from NuMeanFielders/dev/Hal8180

5ab78fe · yesterday

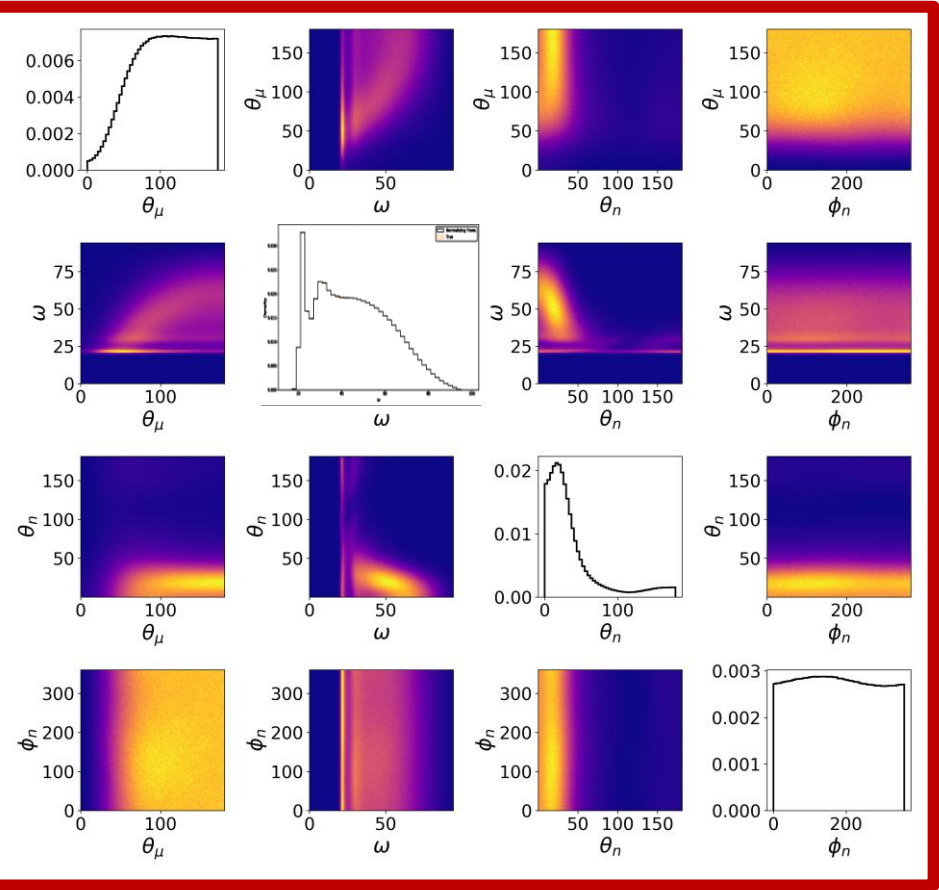
41 Commits

bash	Training for toy and gfg works, added some debugging plo...	2 weeks ago
configs	Improved diagnostic plots	2 weeks ago
docs	Corrected apptainer.def and added .md doc about it	yesterday
env	Adding Apptainer.def for HPC usages	yesterday
src	Removed useless dummy function	last week

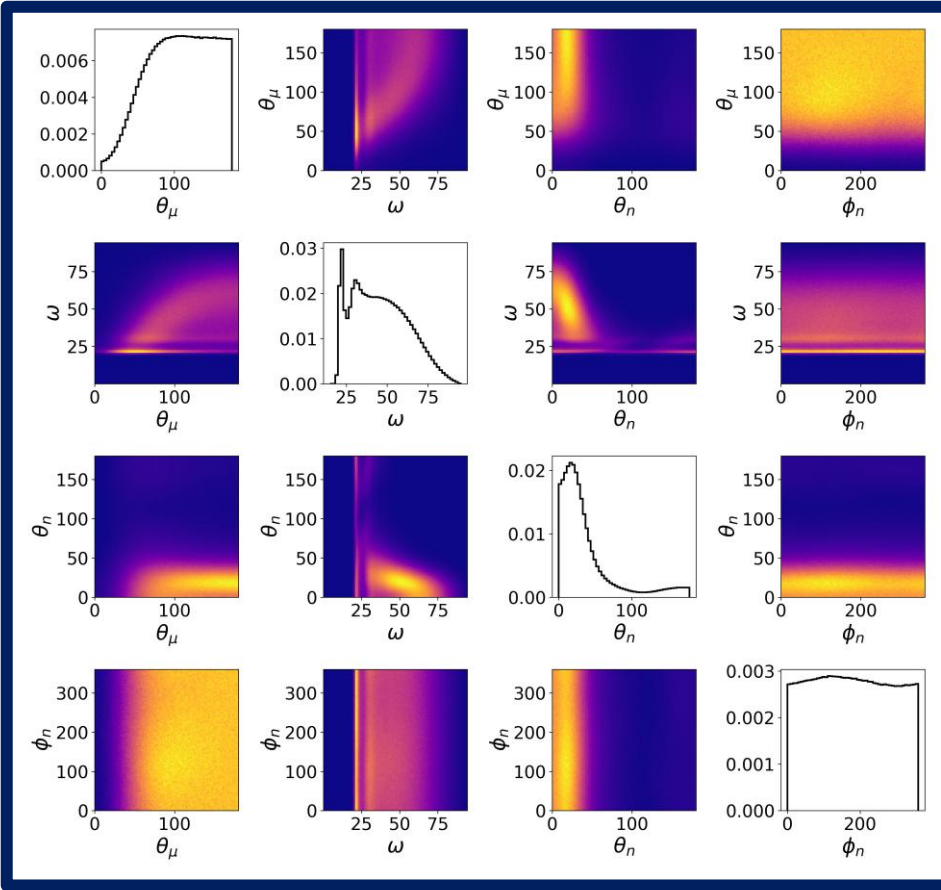
$$E_\nu = 200 \text{ MeV, 1p shell}$$



$E_\nu = 200 \text{ MeV, 1p shell}$

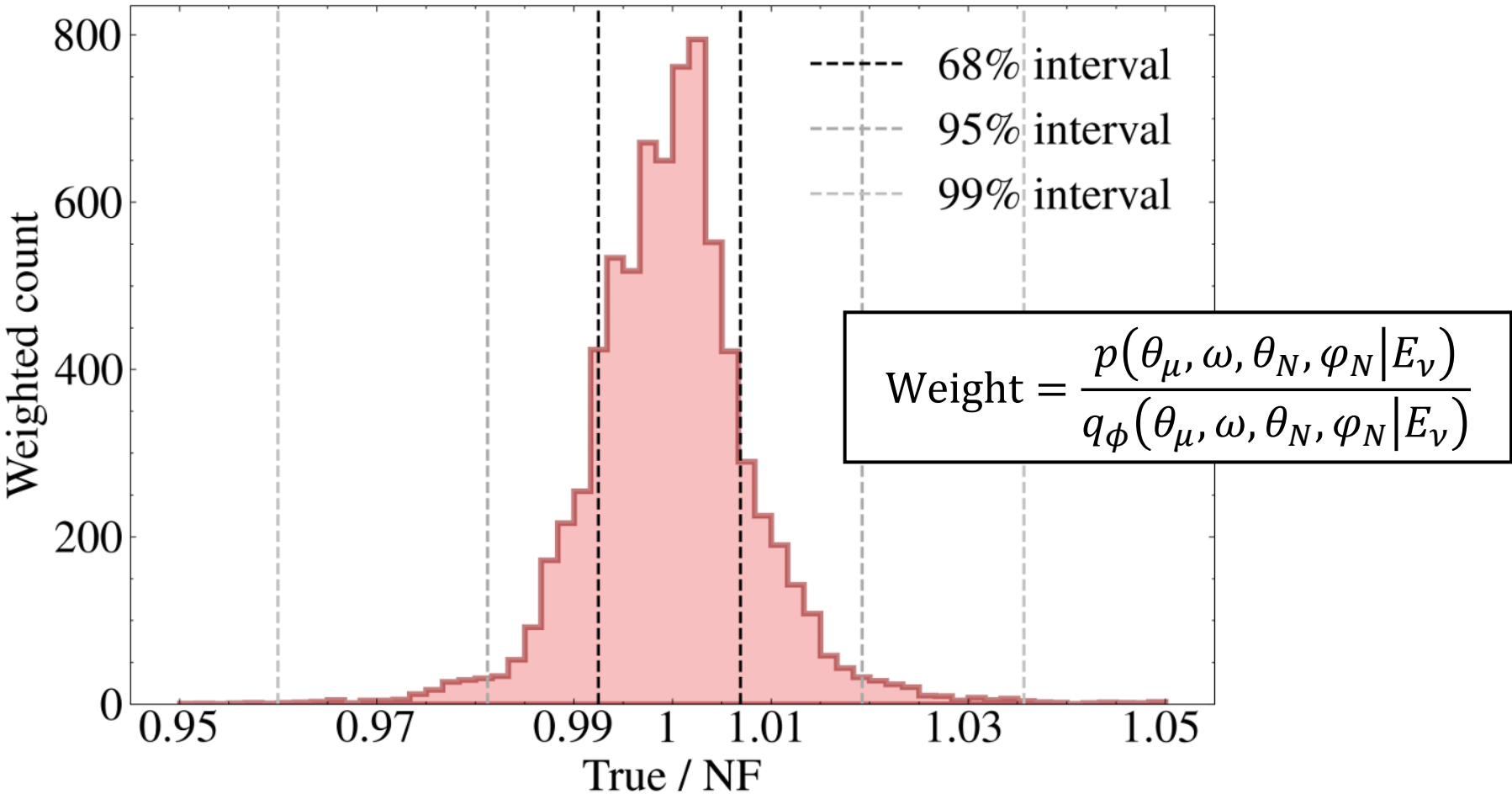


True Cross section



NF Surrogate Cross section

Distribution of weights



*Histogram of weights
(comparing bin contents of size 1 MeV.deg³)*

How to use the NF?*

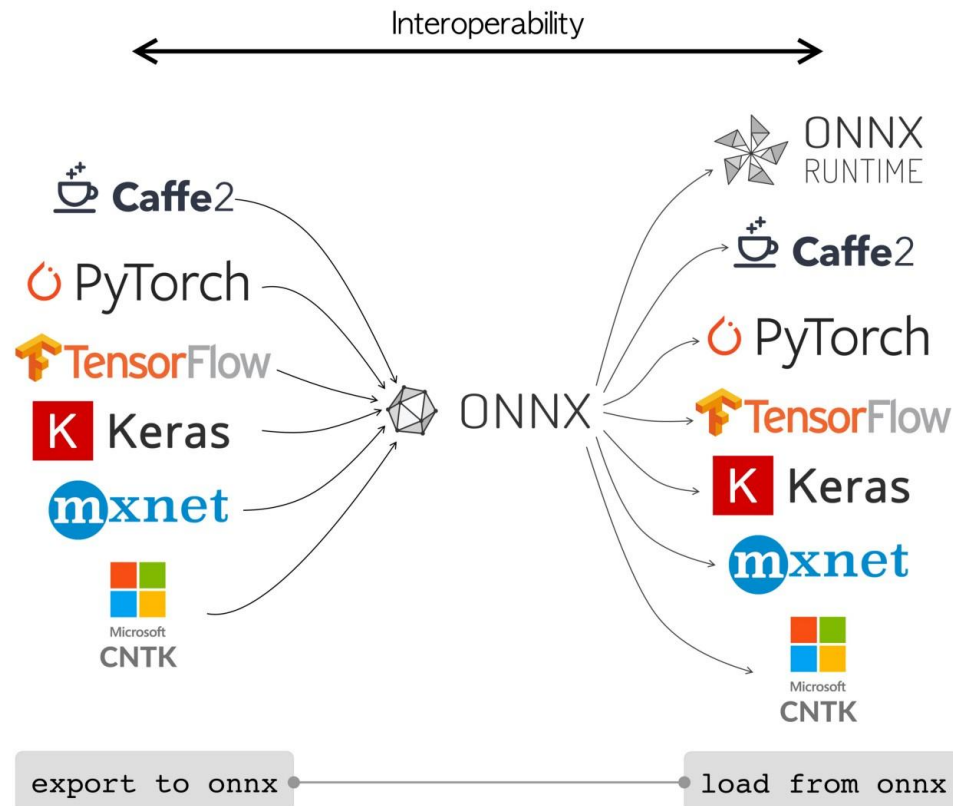
As an importance sampler	
Acceptance rate	>50%
ESS	>98%
Sample quality	Asymptotically unbiased

As a sample emulator	
Samples per day per CPU	~0.1 billion
Samples per day per GPU	~10 billions
Spread of weights **	<1%
Sample quality	Asymptotically biased

* Assuming we want to sample the Ghent CCQE cross section
** Comparing bin content of size 1MeV.deg^3

NEUT interface

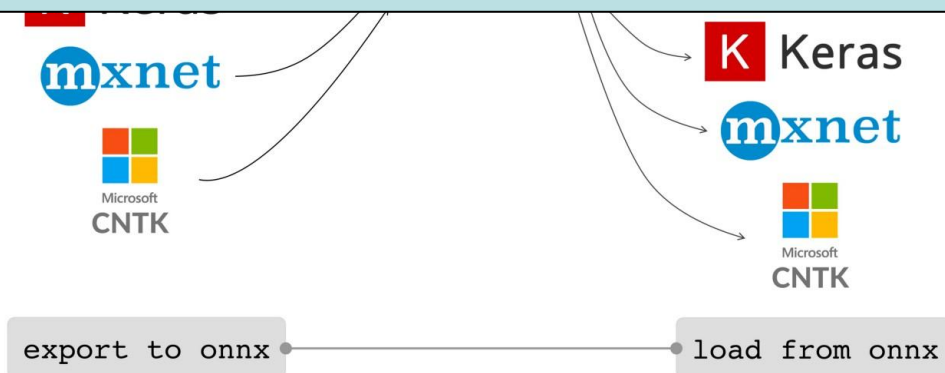
- Models trained in Pytorch (python)
- Models exported to ONNX format (portable graph)
- ONNX Runtime runs inference in C++



NEUT interface

- Models trained in Pytorch (python)
- Models exported to ONNX format (portable graph)
- ONNX Runtime runs inference in C++

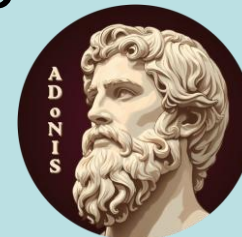
Interfacing NuGeNFlow to any MC generator is now straightforward



NEUT interface

- Models trained in Pytorch (python)
- Models exported to ONNX format (portable graph)
- ONNX Runtime runs inference in C++

Interfacing NuGeNFlow to any MC generator is now straightforward



mxnet



K Keras

mxnet



export to onnx

load from onnx

NEUT interface

- Models trained in Pytorch (python)
- Models exported to ONNX format (portable graph)
- ONNX Runtime runs inference in C++

Interfacing NuGeNFlow to any MC generator is now straightforward



mxnet



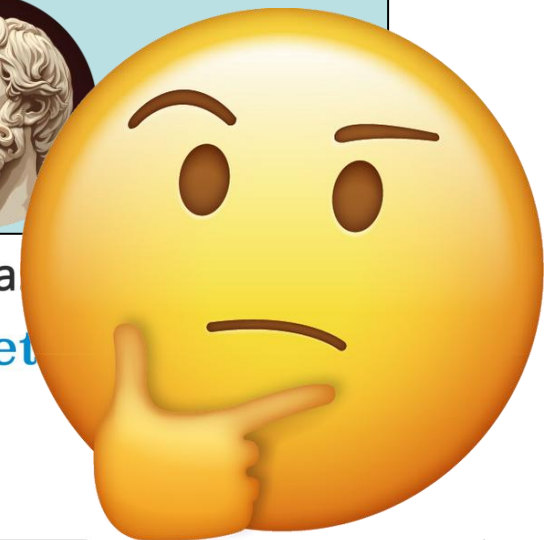
K Keras

mxnet



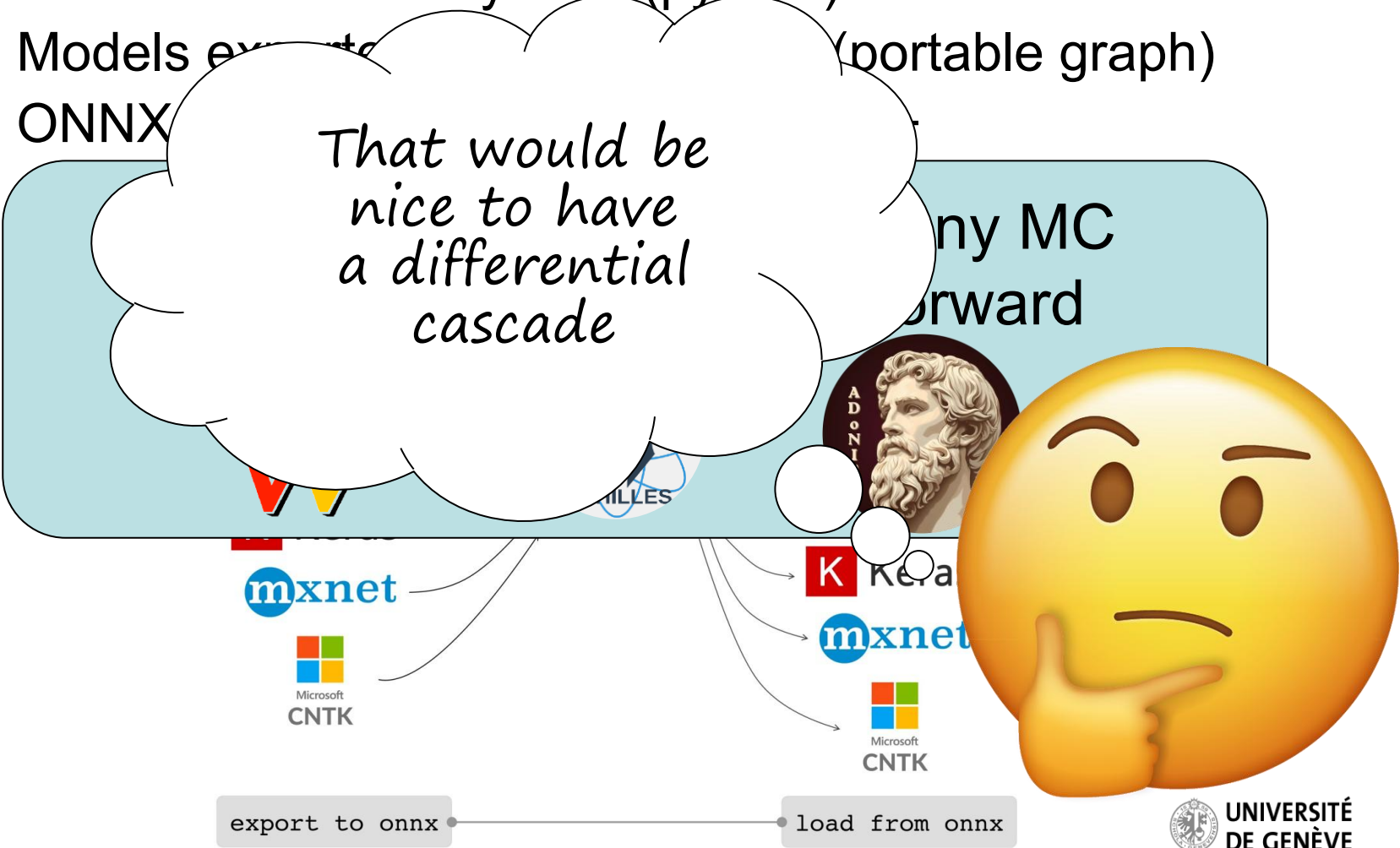
export to onnx

load from onnx



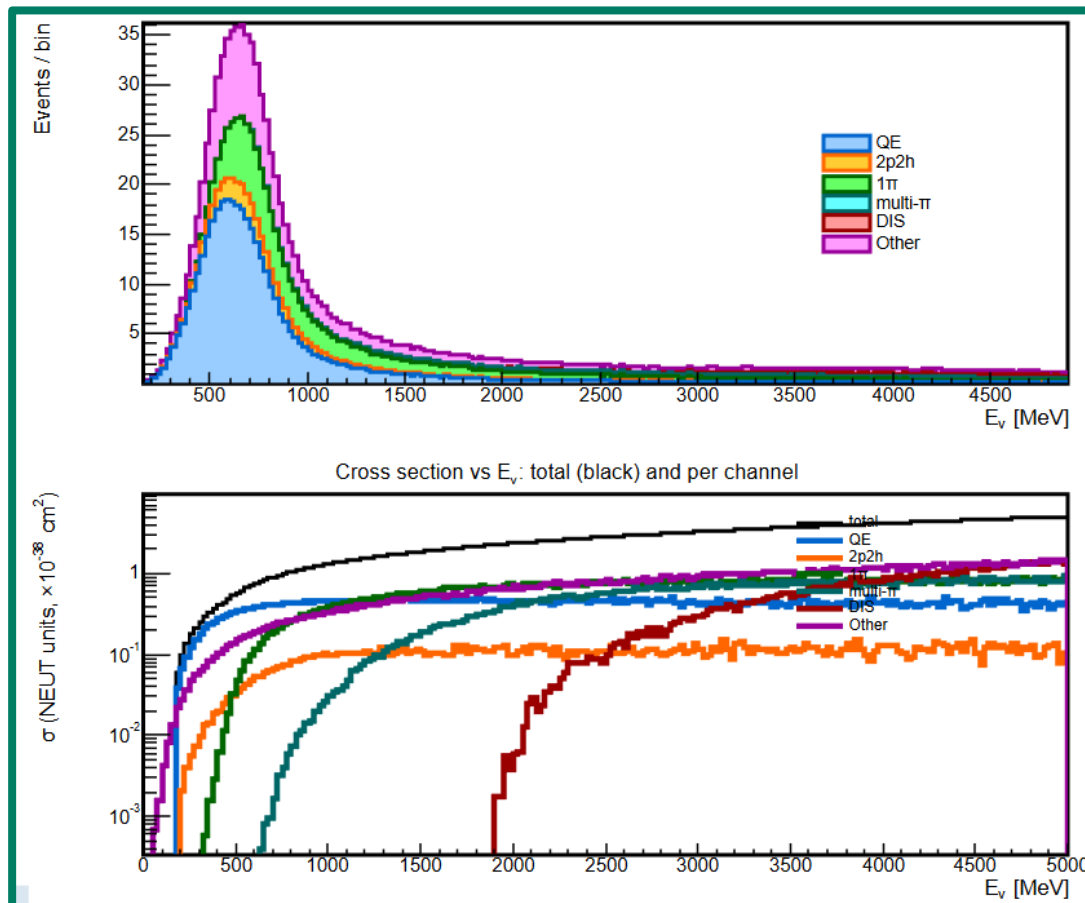
NEUT interface

- Models trained in Pytorch (python)
- Models exported to ONNX (portable graph)
- ONNX



HFMF 1p1h is in NEUT

- 1p1h in the HFMF is fully implemented and available in NEUT
- Will be part of the official NEUT 6.3 tag



*HFMF 1p1h QE
+ standard NEUT channels*

*T2K flux
 ν_μ on C12*

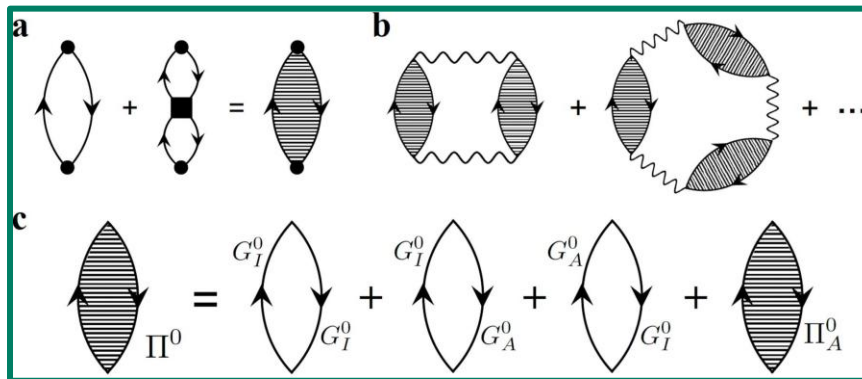
*Event rate (top)
Cross section (bottom)*

Available models

C12 and O16 nuclei

$\nu_\mu, \bar{\nu}_\mu, \nu_e, \bar{\nu}_e$ up to 10 GeV

Available with and without **CRPA**



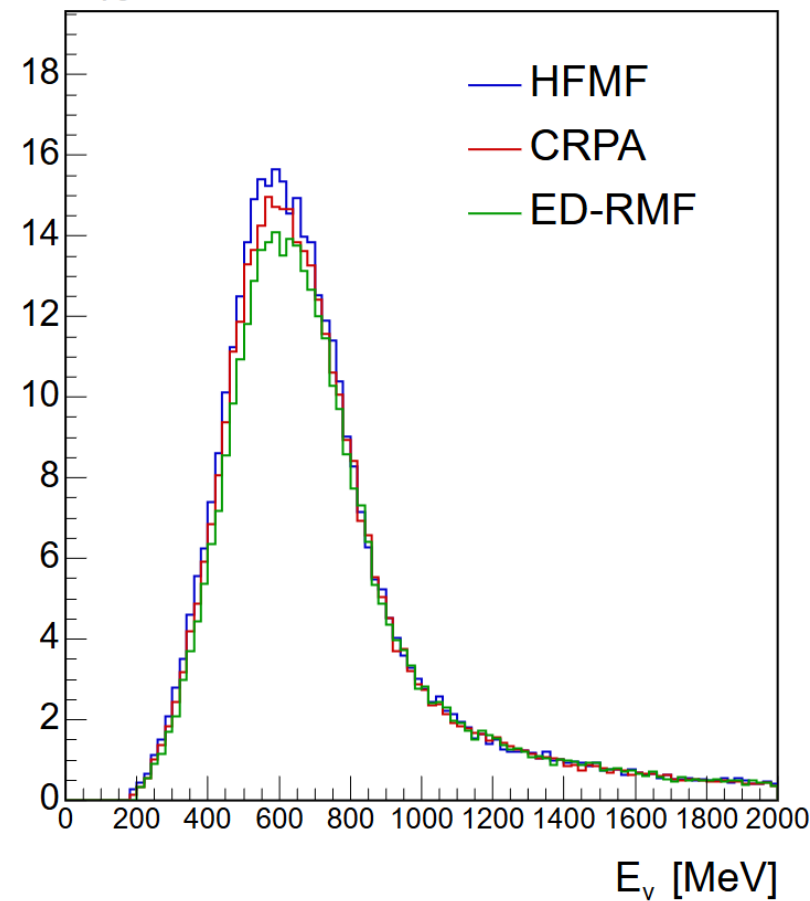
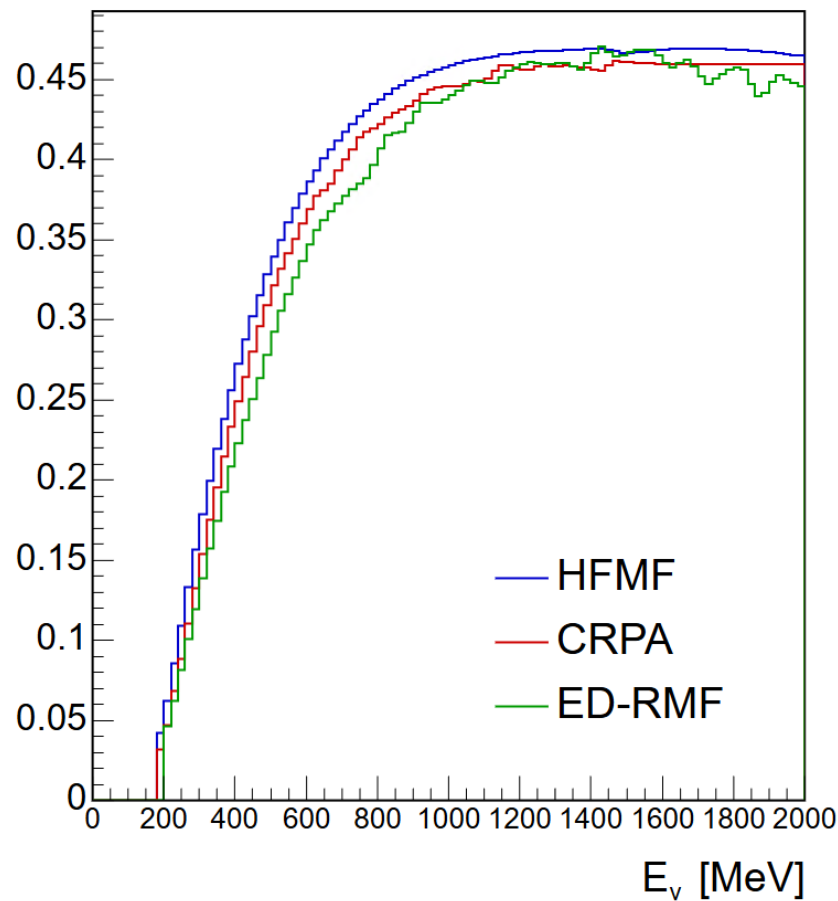
*Dressing of the
particle-hole
responses
(CRPA)*

Specs :

100k/ 5 minutes (Inside NEUT with INC)

400 Mb for all models

Total cross section



*Total cross section per nucleon ($1e-37$) (left)
and event rate for T2K flux (right), ν_μ on C12*

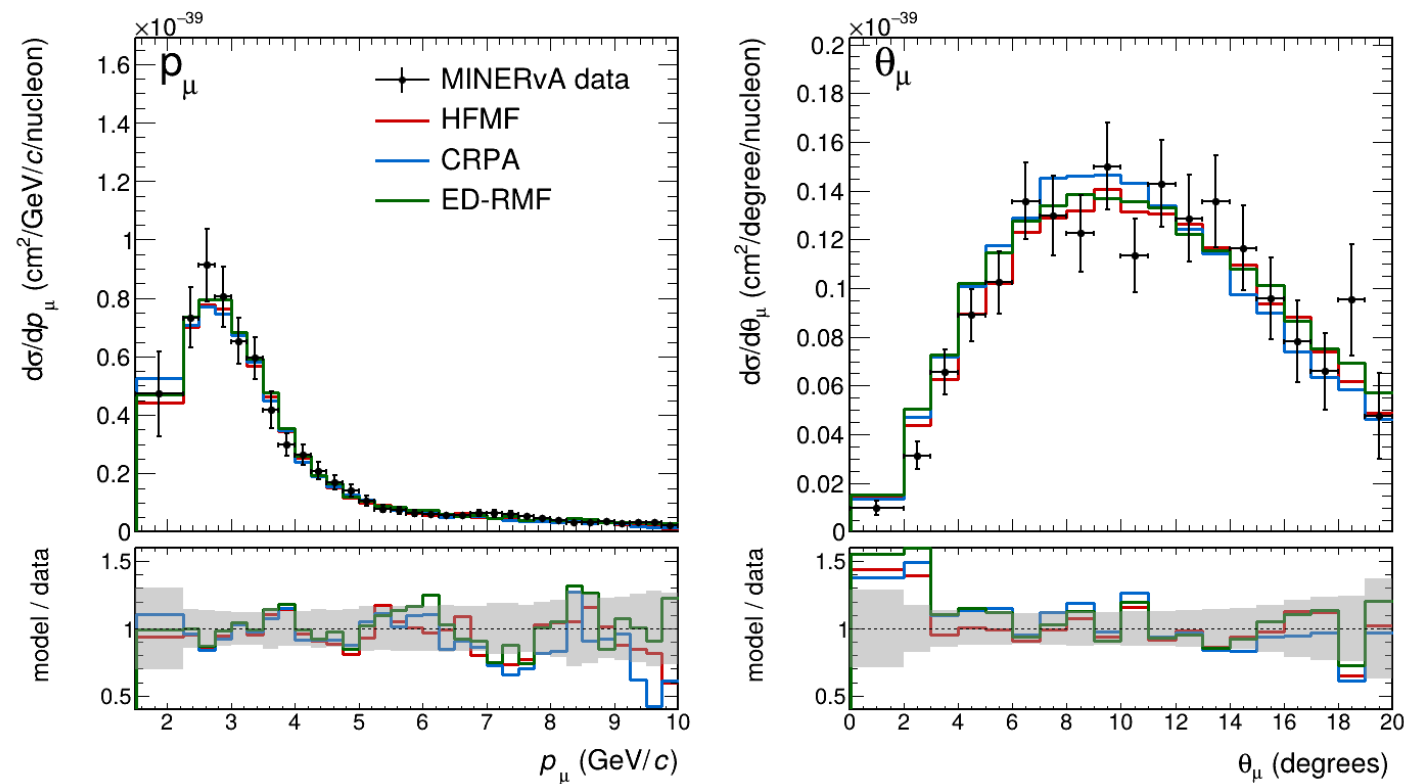
MINERvA comparison

- MINERvA flux-folded, (D'Agostini) unfolded cross-section points
- Hydrocarbon target, NuMI beam
- Data points and covariance matrices from the official release in NUISANCE and compared directly with the truth-level predictions.
- Varied QE (HFMF, HF-CRPA, EDRMF) with other NEUT default channels

	Selection
muon momentum	1500 – 10000 MeV/c
muon angle	< 20° to the beam
proton momentum	450 – 1200 MeV/c
proton angle	< 70°
final state	ν_μ CC, no mesons, ≥ 1 proton

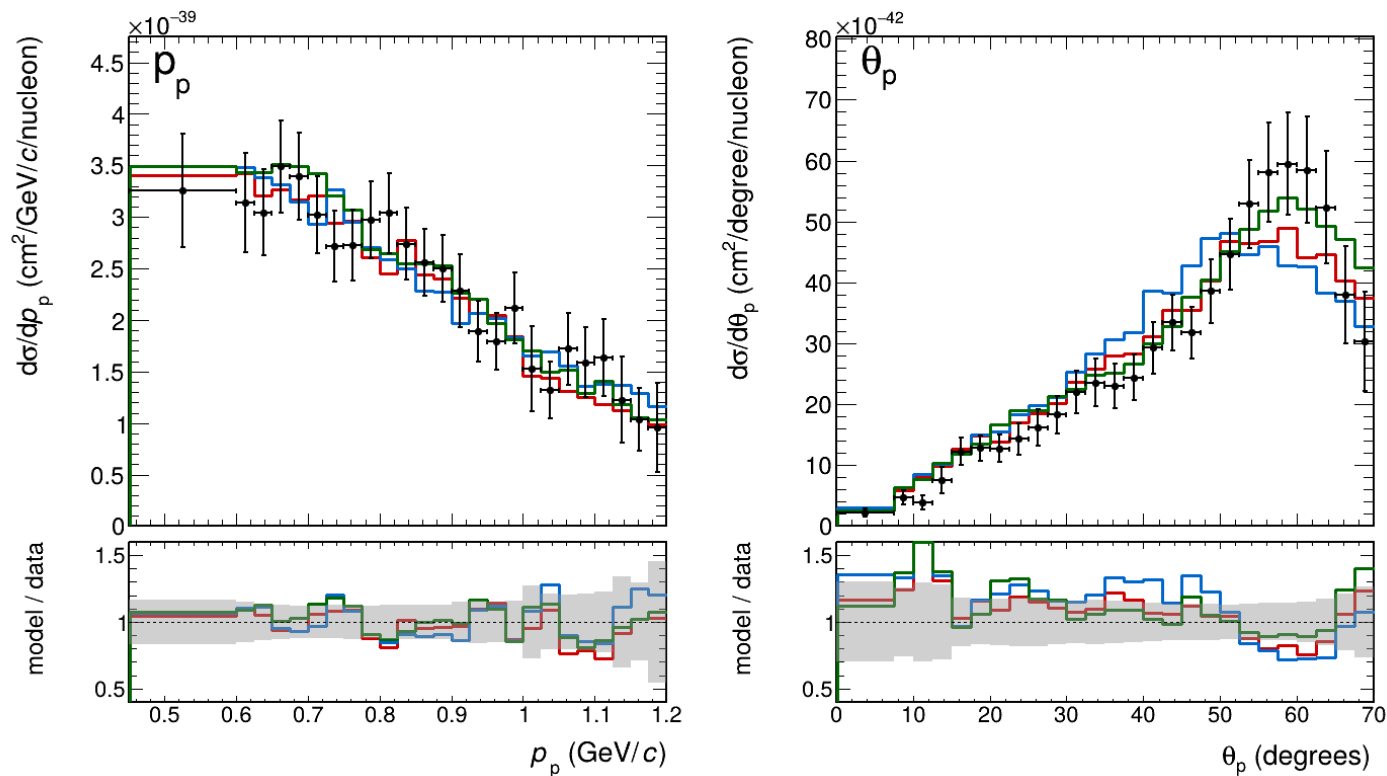
PRL 121, 022504 (2018), reanalysed in PRD 101, 092001 (2020)

Muon kinematics



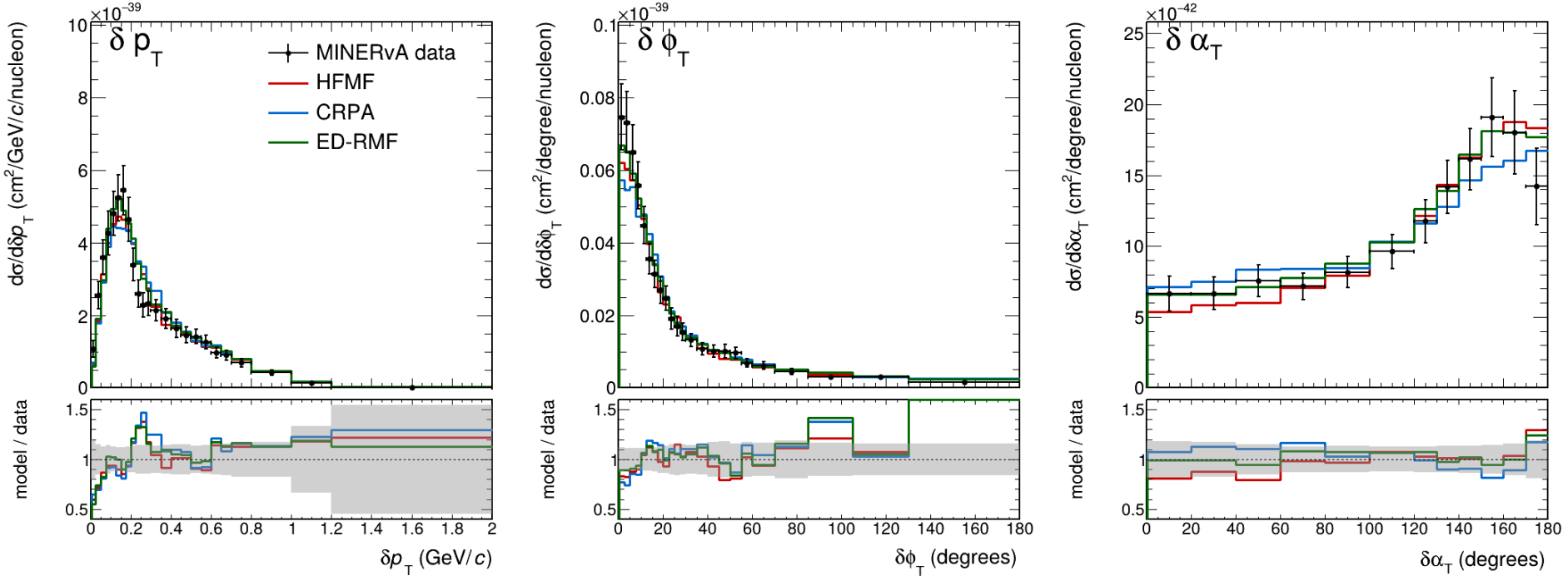
kinematic	ndof	HFMF	ED-RMF	CRPA
p_μ	32	1.21	1.46	1.37
θ_μ	19	1.23	1.78	1.80

Proton kinematics



kinematic	ndof	HFMF	ED-RMF	CRPA
p_p	25	1.18	0.95	2.28
θ_p	26	1.67	1.50	2.71

TKI



Kinematic	ndof	HFMF	ED-RMF	CRPA
δp_T	24	2.55	3.42	4.01
$\delta \phi_T$	23	2.94	2.79	5.21
$\delta \alpha_T$	12	1.66	1.13	1.92
JOINT	161	1.76	1.88	2.76

I. Motivations

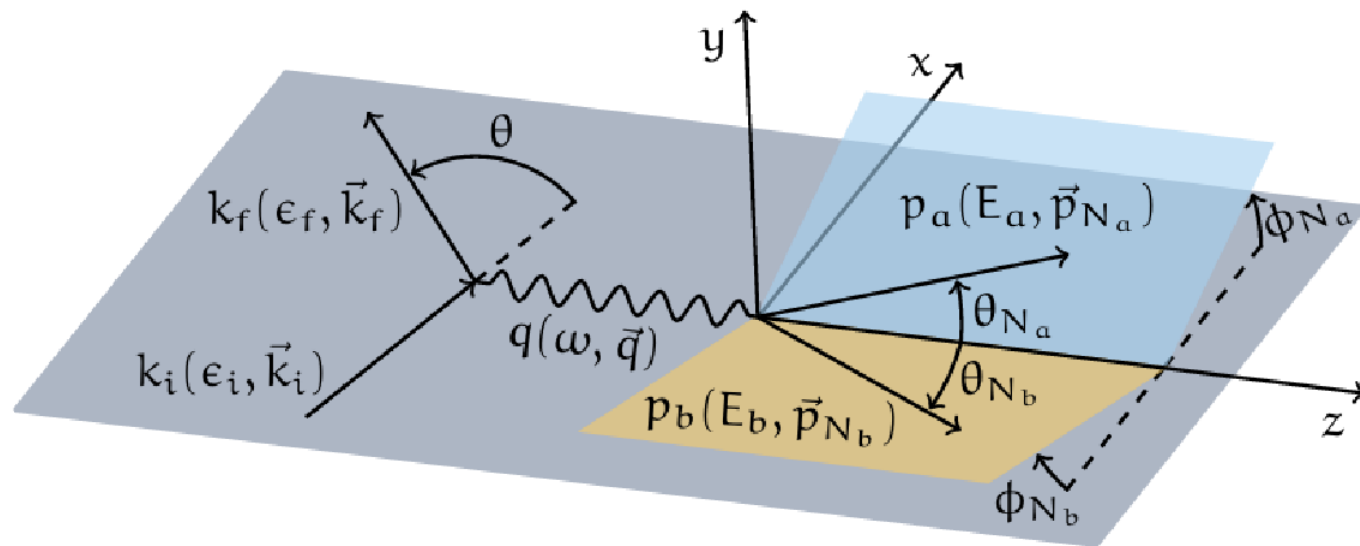
II. Efficient sampling with
Normalizing Flows

III. NuGeNFlow and
its interface to NEUT

IV. Bonus

Exclusive 2p2h in HFMF

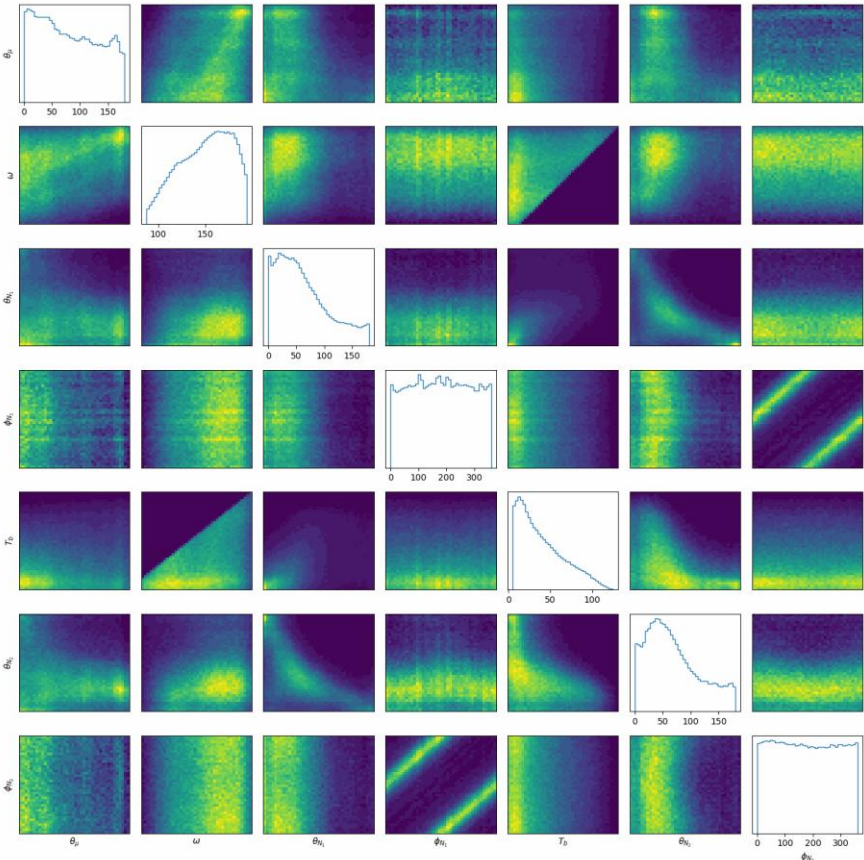
- Weak boson couples to a nucleon pair through two-body currents
- Nucleons in occupied states to the continuum, leaving two holes.
- Bound and emitted nucleons share the same mean-field potential



Schematic of lepton-induced two-nucleon knock-out (Kajetan N.)

Exclusive 2p2h in HFMF

- Weak boson couples to a nucleon pair through two-body currents
- Nucleons in occupied states to the continuum, leaving two holes.
- Bound and emitted nucleons share the same mean-field potential



*Samples from a trained model for the
Mean Field 2p2h exclusive cross
section*

*(pn) configuration with in (1s,1s) shell
For $E\nu \in [300, 1200]$ MeV*

Systematic uncertainties

- Any model relies on theoretical assumptions and approximate fits.
 - Eg, axial form factors, width of the shells etc...
- To regenerate a simulation for different values of systematics, we apply a **precomputed reweight** per event or per bin.
 - We often assume the actions of these parameters are **decorrelated**.

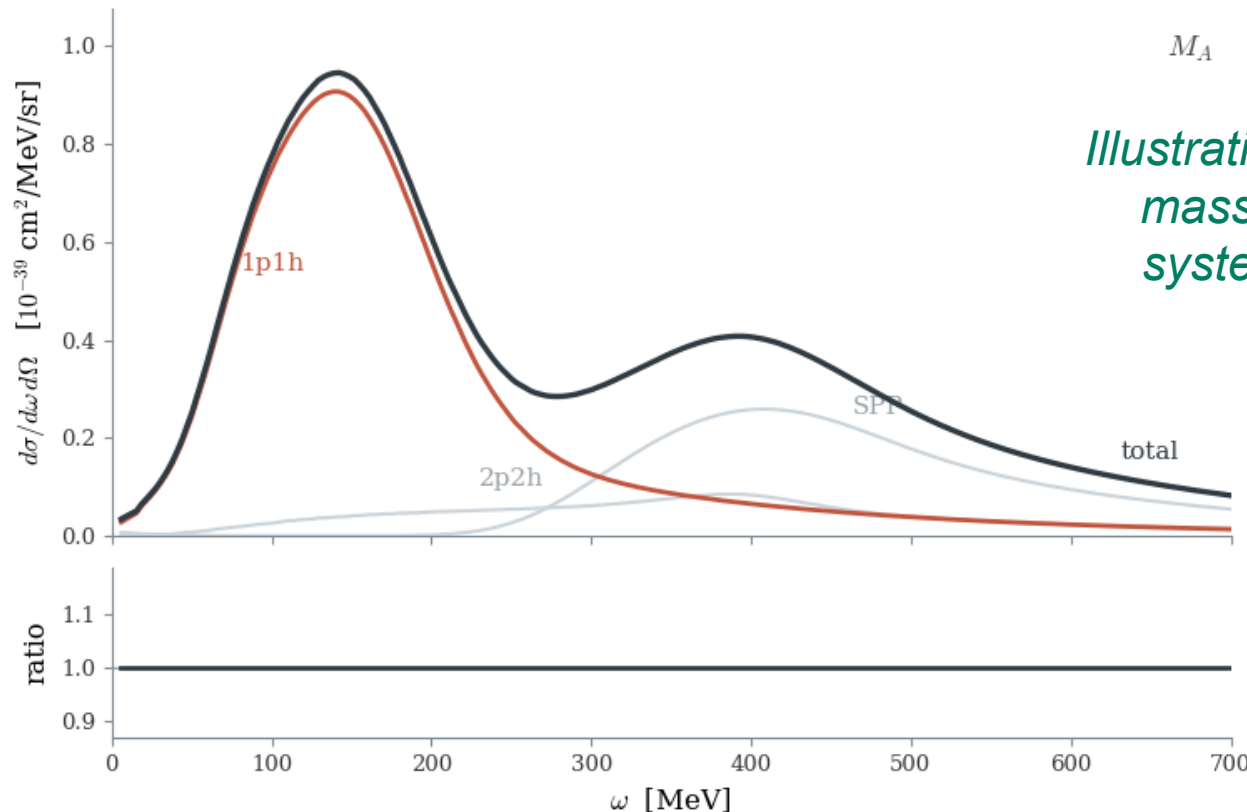


Illustration of the impact of axial mass and in medium delta systematics on the energy transfer profile (SPP fixed here)

Conditional sampler

- Let's call the set of systematics parameters $Y = [Y_1, Y_2 \dots]$
- So far, the differential cross section was only conditioned on E_ν .

$$\frac{\partial^N \sigma_{NF}}{\partial X^N} (X, E_\nu, Y = Y_0) = \underbrace{p_{NF}(X|E_\nu)}_{\text{NF model}} \times \underbrace{f(E_\nu)}_{\substack{\text{Polynomial fit} \\ \text{Of } \sigma(E_\nu)}}$$

Conditional sampler

- Let's call the set of systematics parameters $Y = [Y_1, Y_2 \dots]$
- So far, the differential cross section was only conditioned on E_ν .

$$\frac{\partial^N \sigma_{NF}}{\partial X^N} (X, E_\nu, Y = Y_0) = \underbrace{p_{NF}(X|E_\nu)}_{\text{NF model}} \times \underbrace{f(E_\nu)}_{\text{Polynomial fit Of } \sigma(E_\nu)}$$

- To build a Y -dependent sampler, we need two ingredients :
 1. A total cross section for any Y : $\sigma(E_\nu, Y)$
 2. A conditional sampler: $p(X|E_\nu, Y)$

Conditional sampler

- Let's call the set of systematics parameters $Y = [Y_1, Y_2 \dots]$
- So far, the differential cross section was only conditioned on E_ν .

$$\frac{\partial^N \sigma_{NF}}{\partial X^N} (X, E_\nu, Y = Y_0) = \underbrace{p_{NF}(X|E_\nu)}_{\text{NF model}} \times \underbrace{f(E_\nu)}_{\text{Polynomial fit Of } \sigma(E_\nu)}$$

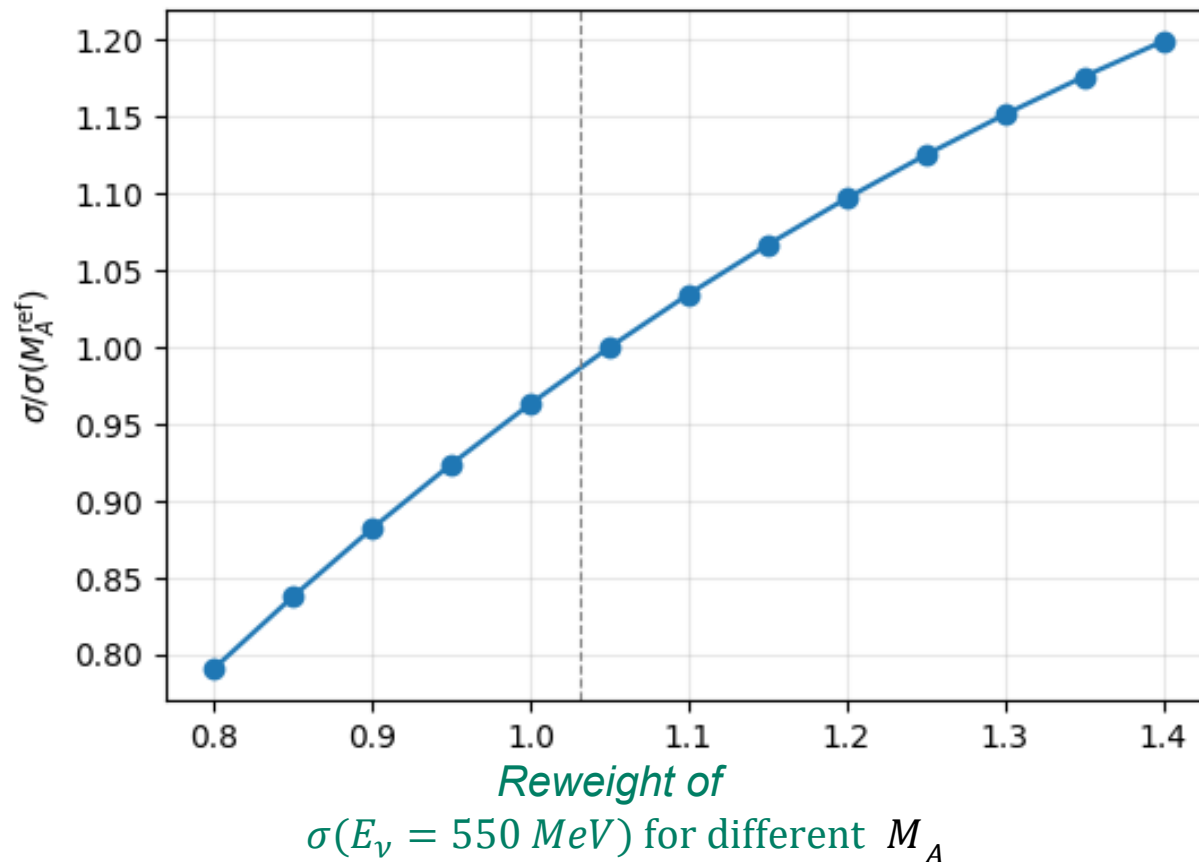
- To build a Y -dependent sampler, we need two ingredients :
 1. A total cross section for any Y : $\sigma(E_\nu, Y)$
 2. A conditional sampler: $p(X|E_\nu, Y)$

$$\frac{\partial^N \sigma_{NF}}{\partial X^N} (X, E_\nu, Y) = \underbrace{p_{NF_1}(X|E_\nu, Y)}_{\text{Conditional NF model}} \times \underbrace{p_{NF_2}(Y|E_\nu) \times f(E_\nu)}_{\sigma(E_\nu, Y)}$$

Conditional parameters in 2p2h

Axial form factor

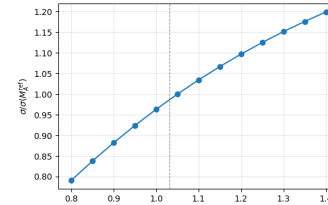
- Smooth parameter : Axial mass
- Can be extended to other parametrizations (z-expansion ...)



Conditional parameters in 2p2h

Axial form factor

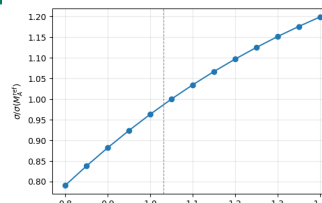
- Smooth parameter : Axial mass
- Can be extended to other parametrizations (z-expansion ...)



Conditional parameters in 2p2h

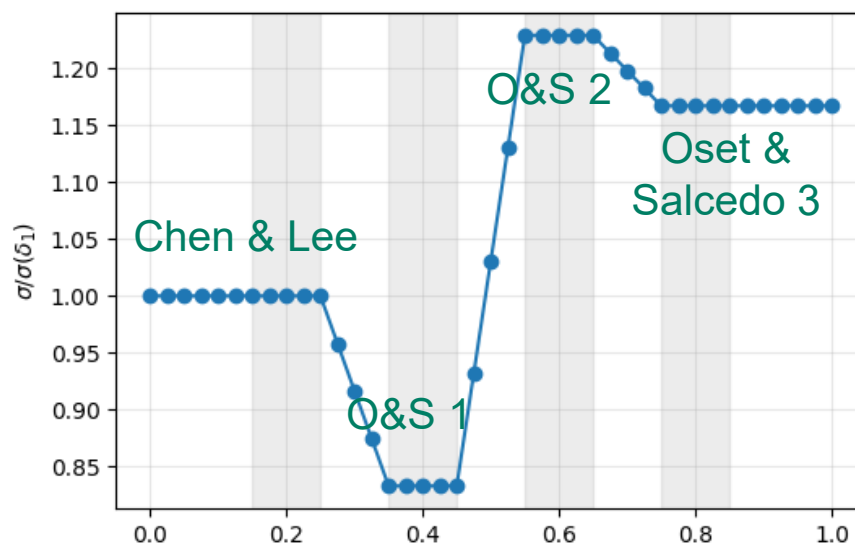
Axial form factor

- Smooth parameter : Axial mass
- Can be extended to other parametrizations (z-expansion ...)



In medium Δ propagator

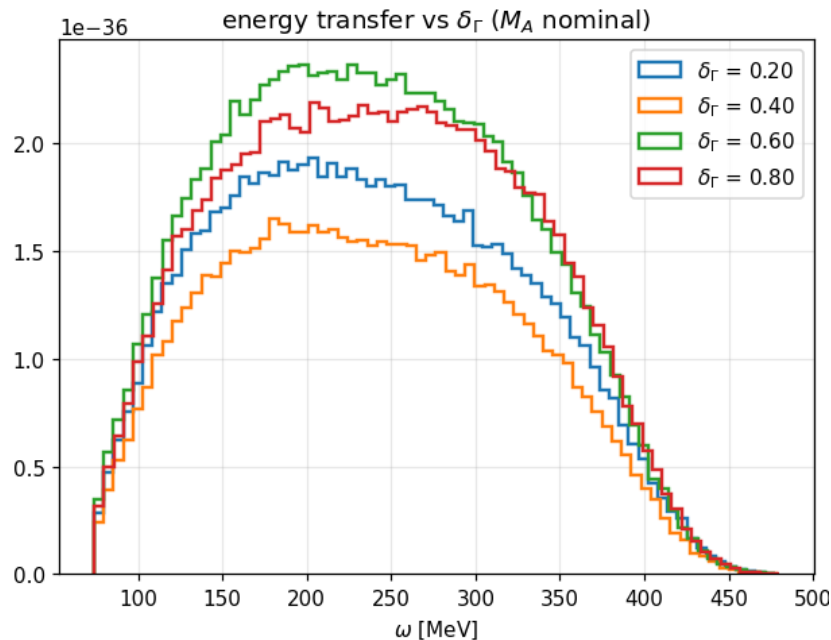
- Different prescriptions for the Δ propagator are available.
- They change the real and imaginary part of its self-energy.
- Discrete choices need to be made continuous



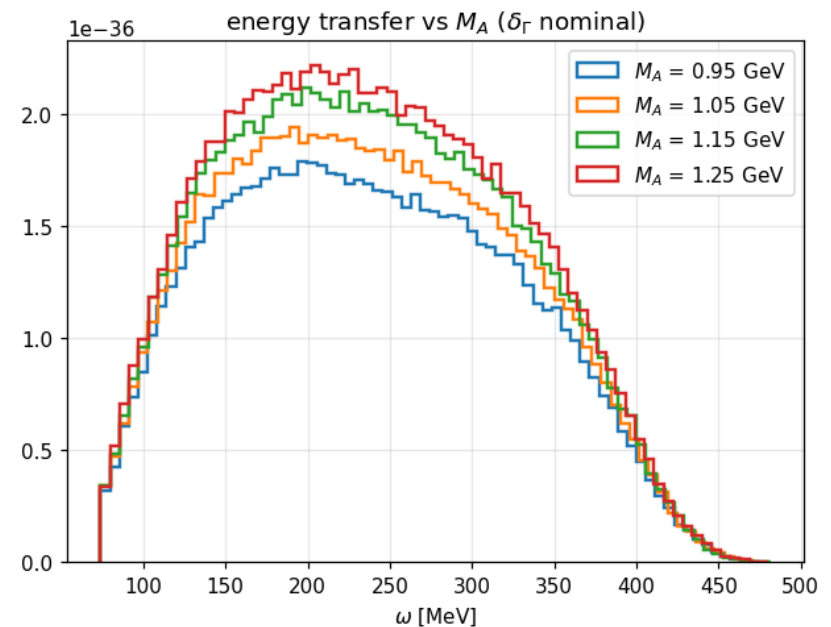
Reweight of $\sigma(E_\nu = 550 \text{ MeV})$ for different Δ prescriptions

Conditional sampling

- Sampling trained conditional NF (fixed isospins and shells)
- Flat flux between 550 and 650 MeV (Same POT)



*Energy transfer distribution for
different in medium
 Δ prescriptions*



*Energy transfer distribution for
different M_A values*

Missing points

- Any model relies on theoretical assumptions and approximate fits.
 - Eg, axial form factors, width of the shells etc...
- To regenerate a simulation for different values of systematics, we apply a **precomputed reweight** per event or per bin.
 - **Precomputed reweight** per event or per bin
 - We often assume the actions of these parameters are **decorrelated**.

Conclusion

- **How are used Normalizing Flows ?**
 - Efficient sampling of the kinematic space
 - Importance/Rejection sampling or as a surrogate
- **What are the properties of Normalizing Flows ?**
 - Fast sampling and density evaluation
 - Light
 - Differentiable on kinematics, neutrino energy and model knobs
- **What the software does ?**
 - Train and sample NF in the context of exclusive neutrino-nucleus cross sections
 - Focus on time consuming high dimensional cross-sections
 - Private repo on GitHub but soon public, 1p1h and 2p2h available, soon SPP

Backup

Event generation with NF in HEP

Event Generation with Normalizing Flows

Christina Gao,¹ Stefan Hoche,¹ Joshua Isaacson,¹ Claudius Krause,¹ and Holger Schulz²

¹Fermi National Accelerator Laboratory, Batavia, IL, 60510, USA

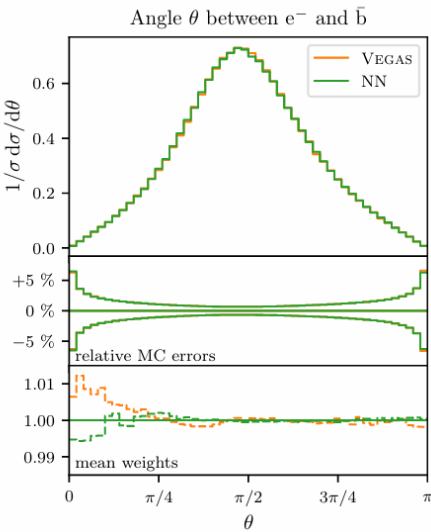
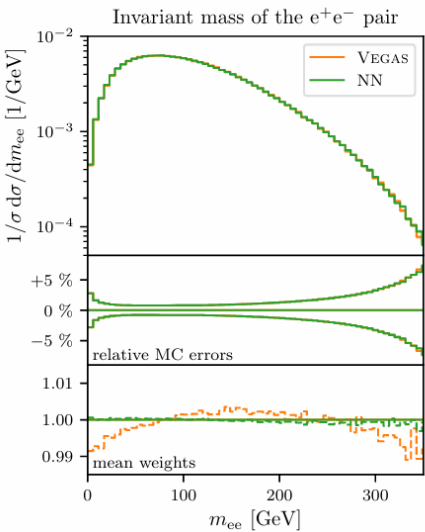
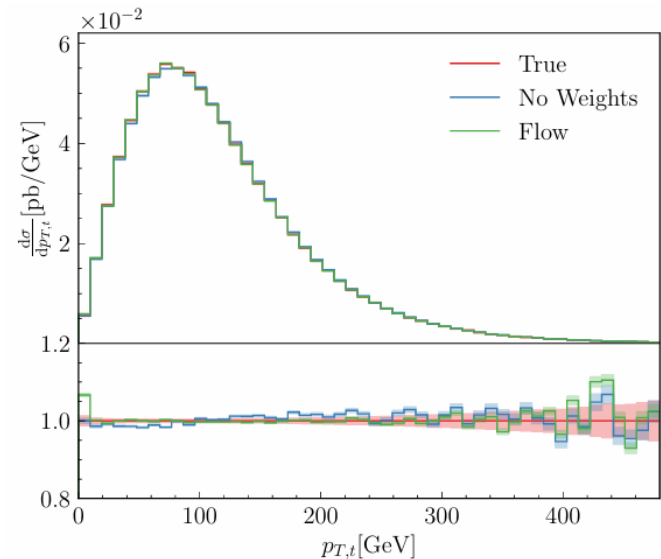
²Department of Physics, University of Cincinnati, Cincinnati, OH 45219, USA

We present a novel integrator based on normalizing flows which can be used to improve the unweighting efficiency of Monte Carlo event generators for collider physics simulations. In contrast to machine learning approaches based on surrogate models, our method generates the correct result even if the underlying neural networks are not optimally trained. We exemplify the new strategy using the example of Drell-Yan type processes at the LHC, both at leading and partially at next-to-leading order QCD.

$$pp \rightarrow W/Z + n_j$$

(Gao et al.)

2001.10028, PRD



Top-quark pair production
(Bothmann et al.)

<https://arxiv.org/pdf/2203.07460>

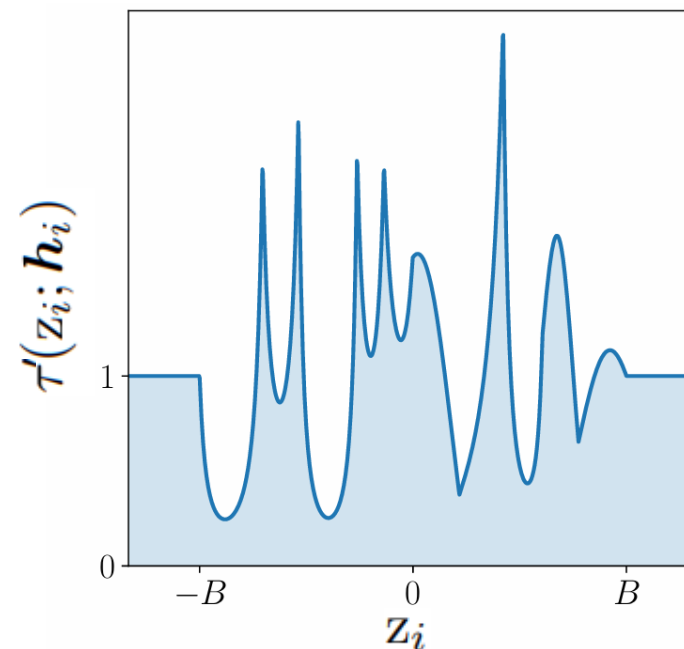
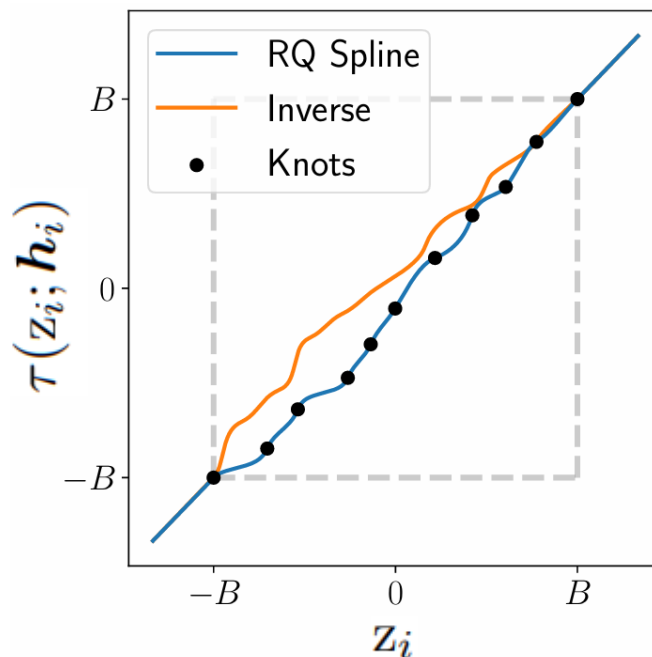
Top pair production events
(Stienen et al.)

<https://scipost.org/SciPostPhys.10.2.038/pdf>

Rational quadratic Neural Spline Flows

$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i})$$

- Each transformer is parametrized by a **monotonous spline made of piecewise rational quadratic functions**.
- One of the **most expressive** flows



RQ-NSF and its derivative

Autoregressive flows

Computation of $|\det(J_T(\mathbf{u}))|^{-1}$ is expensive ($\sim O(D^3)$)

$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i})$$

1D transformation

Conditioner “The NN”

$$J_{f_\phi}(\mathbf{z}) = \begin{bmatrix} \frac{\partial \tau}{\partial z_1}(z_1; \mathbf{h}_1) & & & \mathbf{0} \\ & \ddots & & \\ & & \mathbf{L}(\mathbf{z}) & \\ & & & \frac{\partial \tau}{\partial z_D}(z_D; \mathbf{h}_D) \end{bmatrix}$$

Triangular Jacobian

$$\log |\det J_{f_\phi}(\mathbf{z})| = \log \left| \prod_{i=1}^D \frac{\partial \tau}{\partial z_i}(z_i; \mathbf{h}_i) \right| = \sum_{i=1}^D \log \left| \frac{\partial \tau}{\partial z_i}(z_i; \mathbf{h}_i) \right|$$

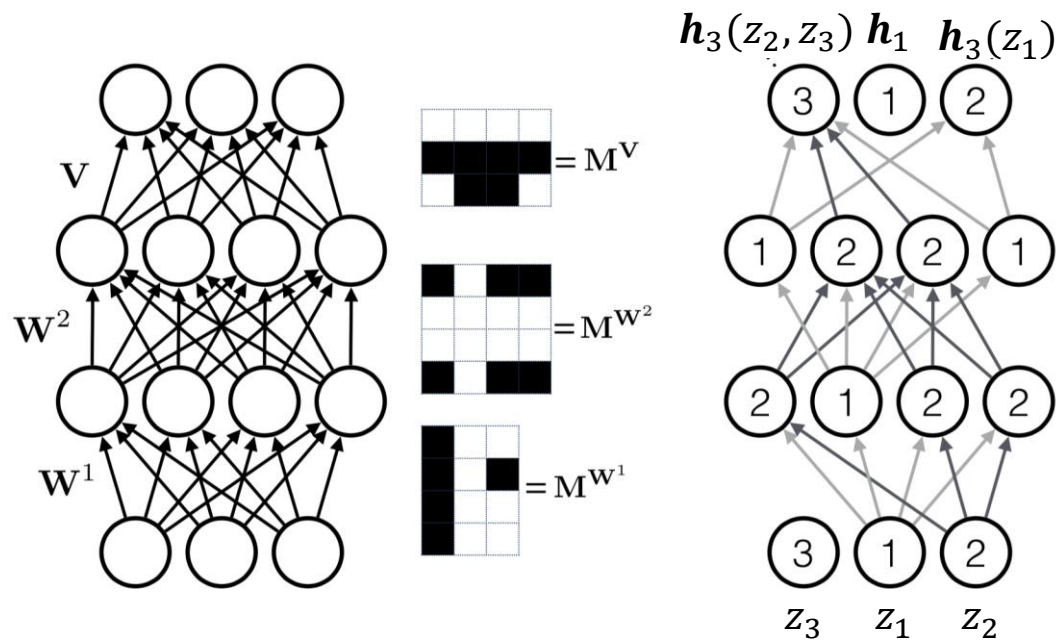
Autoregressive flows implementation

The NN implementation is **easy** !

It is a **standard neural network with masked connections**

What we want to know is the “distortion at a given point z ”

$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i})$$



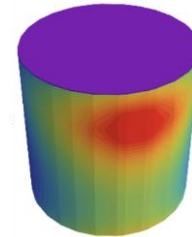
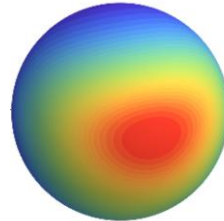
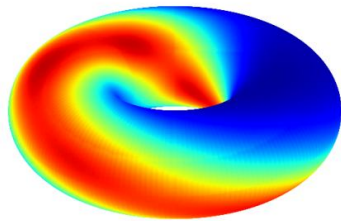
Masked Autoregressive Network

NF on non-Euclidean manifolds

- Differential cross sections depend on angular quantities
- Need to be defined on their natural spaces (circle, tori, sphere ...)

Normalizing Flows on Tori and Spheres

Danilo Jimenez Rezende^{*1} George Papamakarios^{*1} Sébastien Racanière^{*1} Michael S. Albergo²
 Gurtej Kanwar³ Phiala E. Shanahan³ Kyle Cranmer²

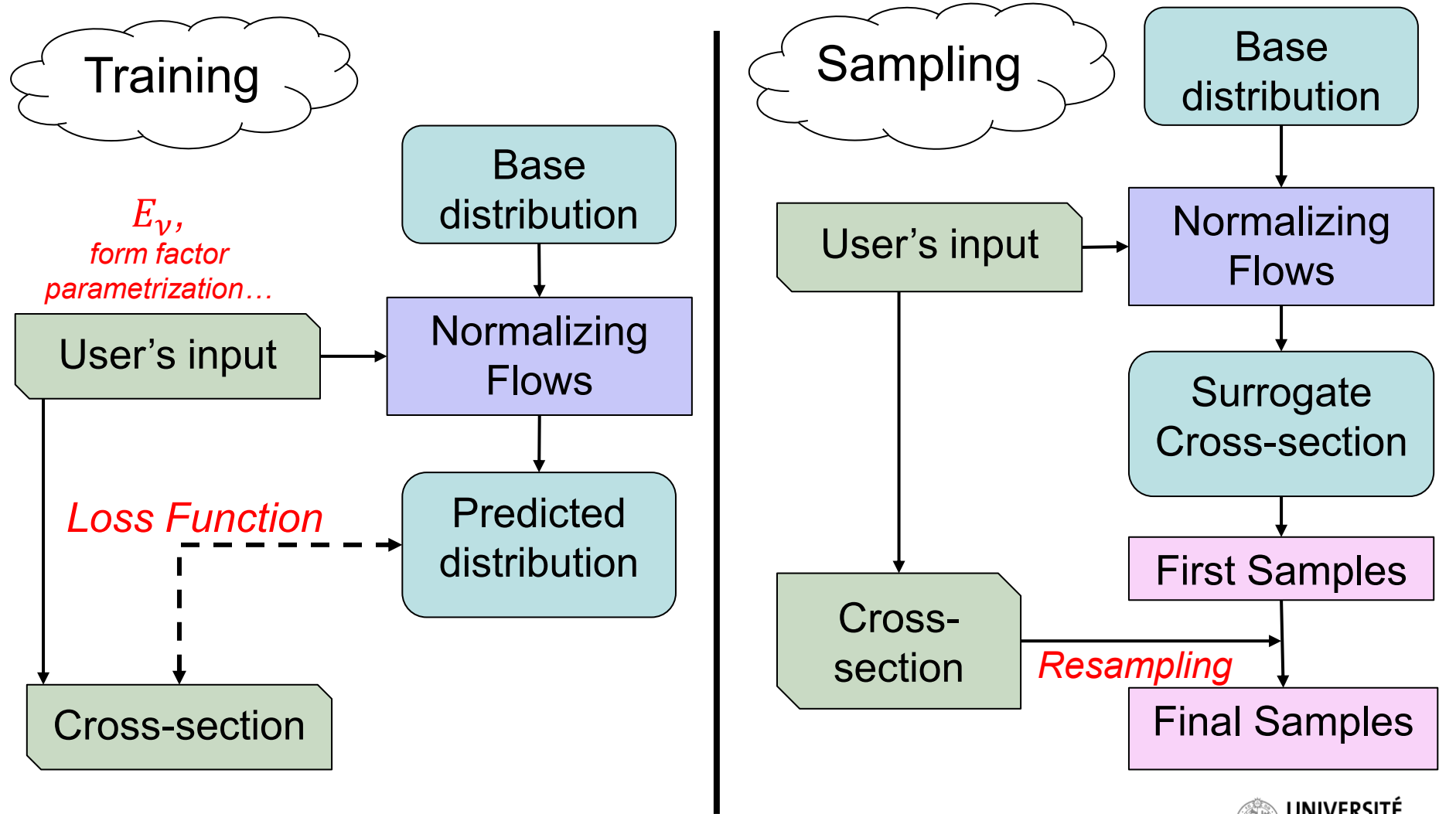


<https://arxiv.org/pdf/2002.02428>

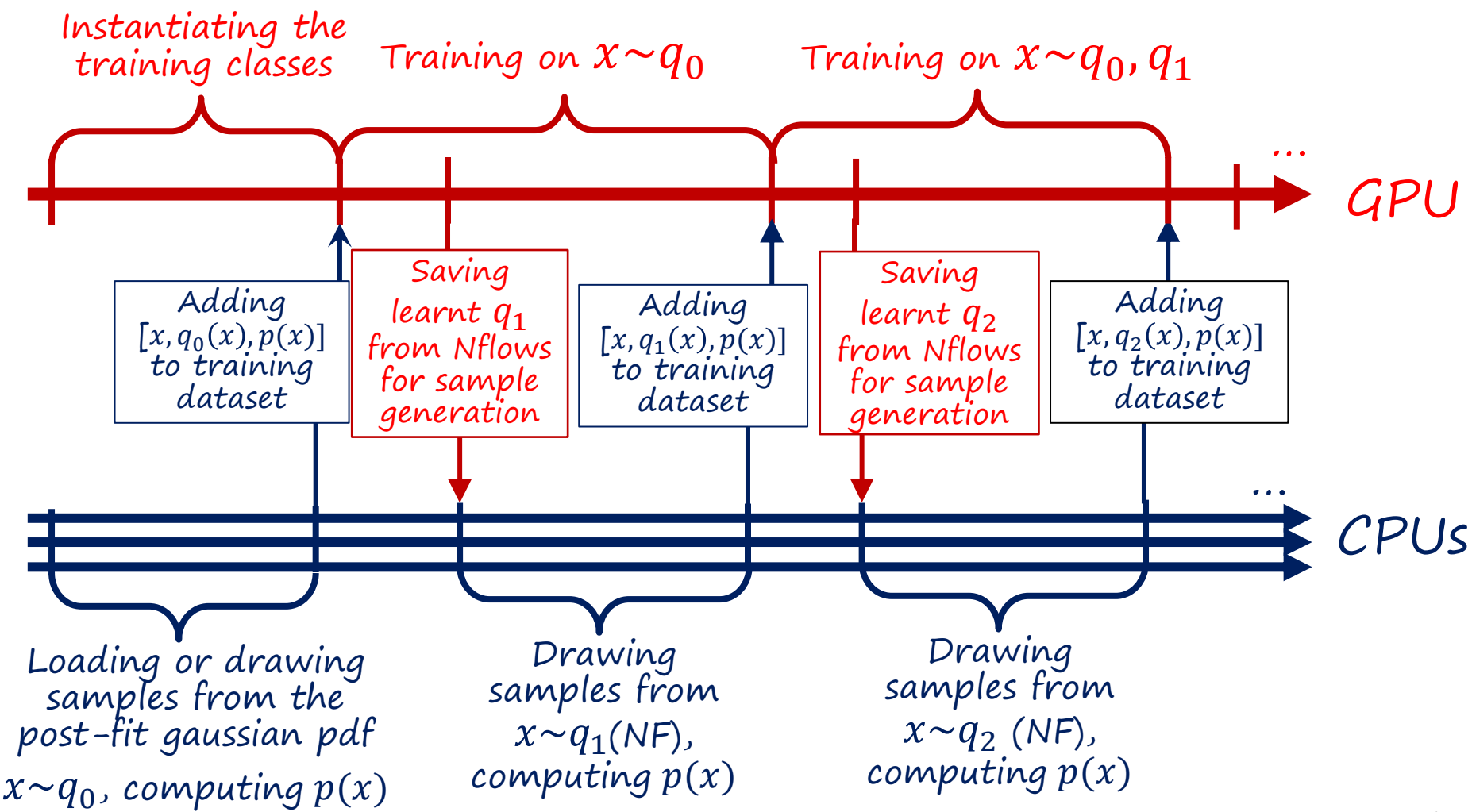
- The two inclusive kinematics (ω, θ_μ) can be represented on a **cylinder** $\mathcal{C}$ and the two nucleon angles (θ_N, ϕ_N) on a **sphere** S^2

$$\text{Manifold } \mathcal{M} = \mathcal{C} \times S^2$$

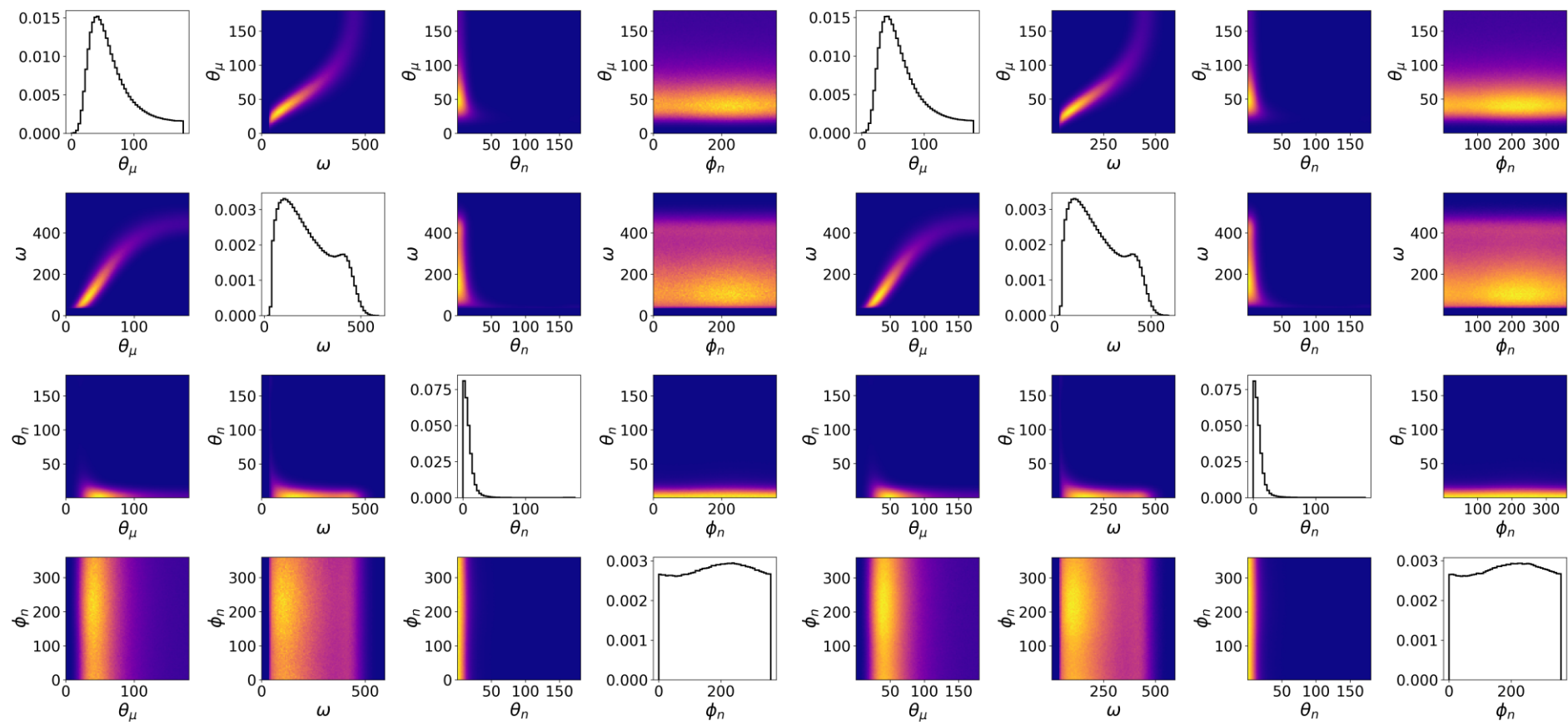
Training and sampling process



Training/sampling process



700 MeV, 1s shell



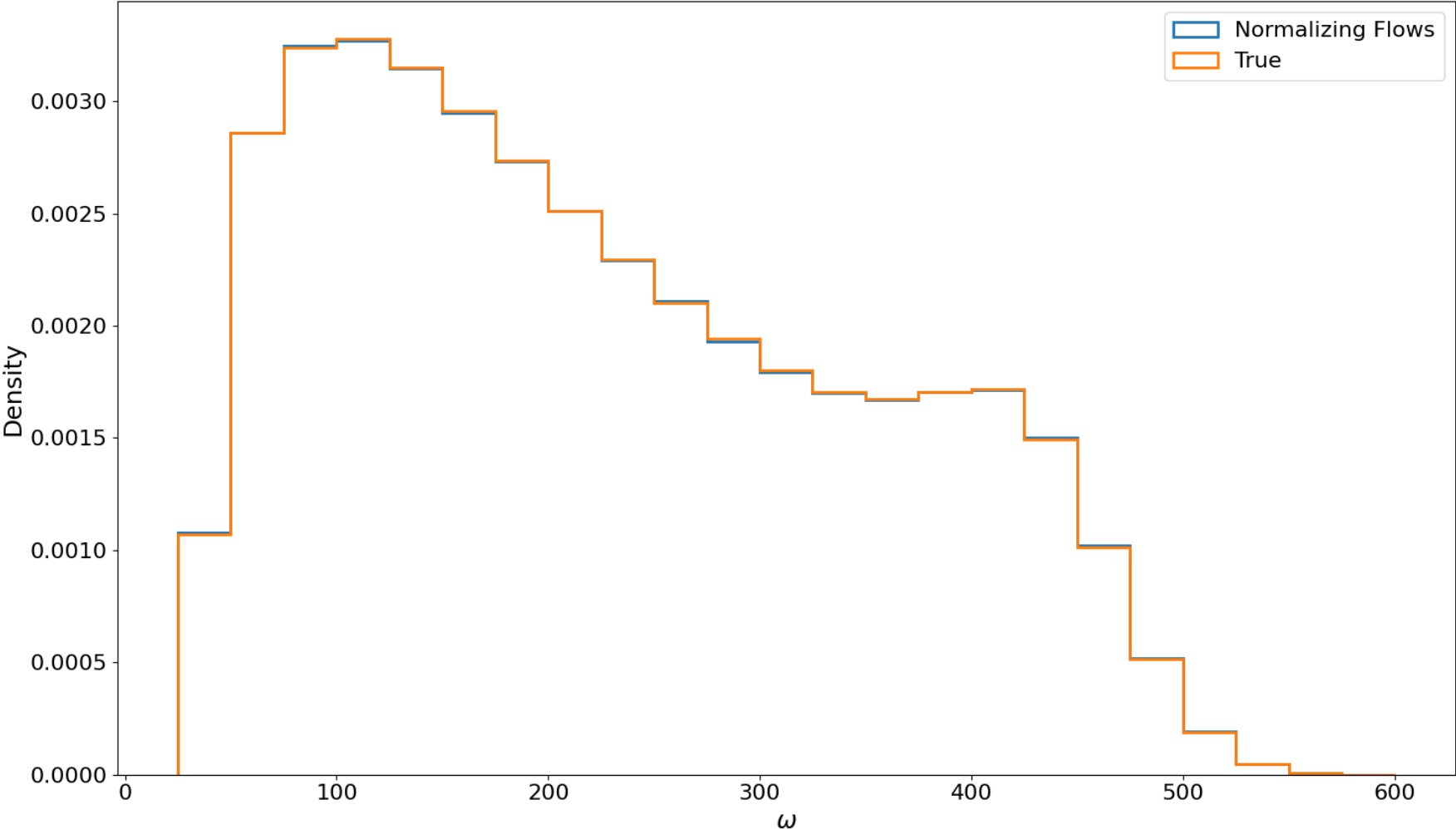
True Cross section

NF Surrogate Cross section



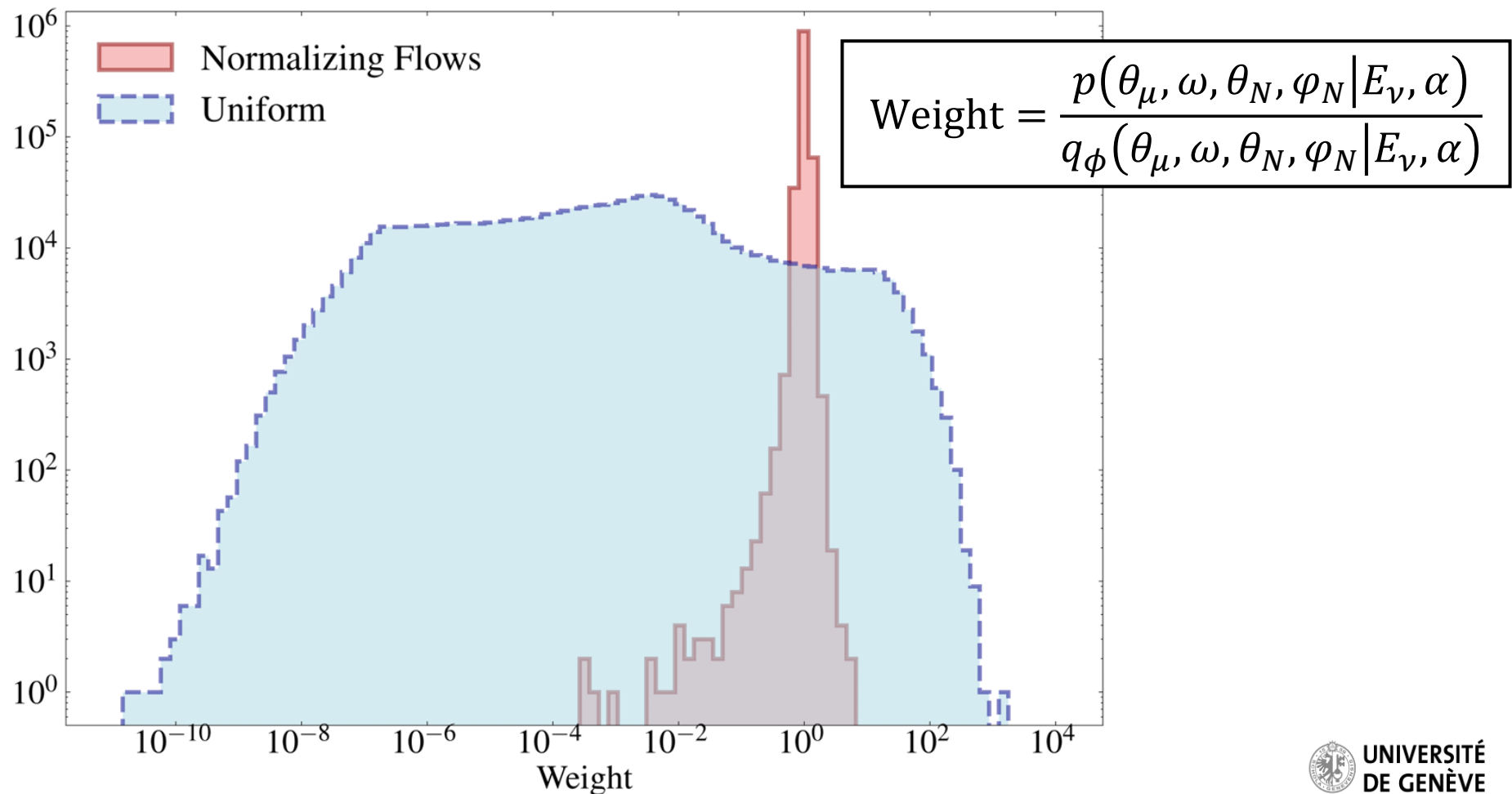
700 MeV, 1s shell

$E_\nu = 700 \text{ MeV} - \omega$ Distribution



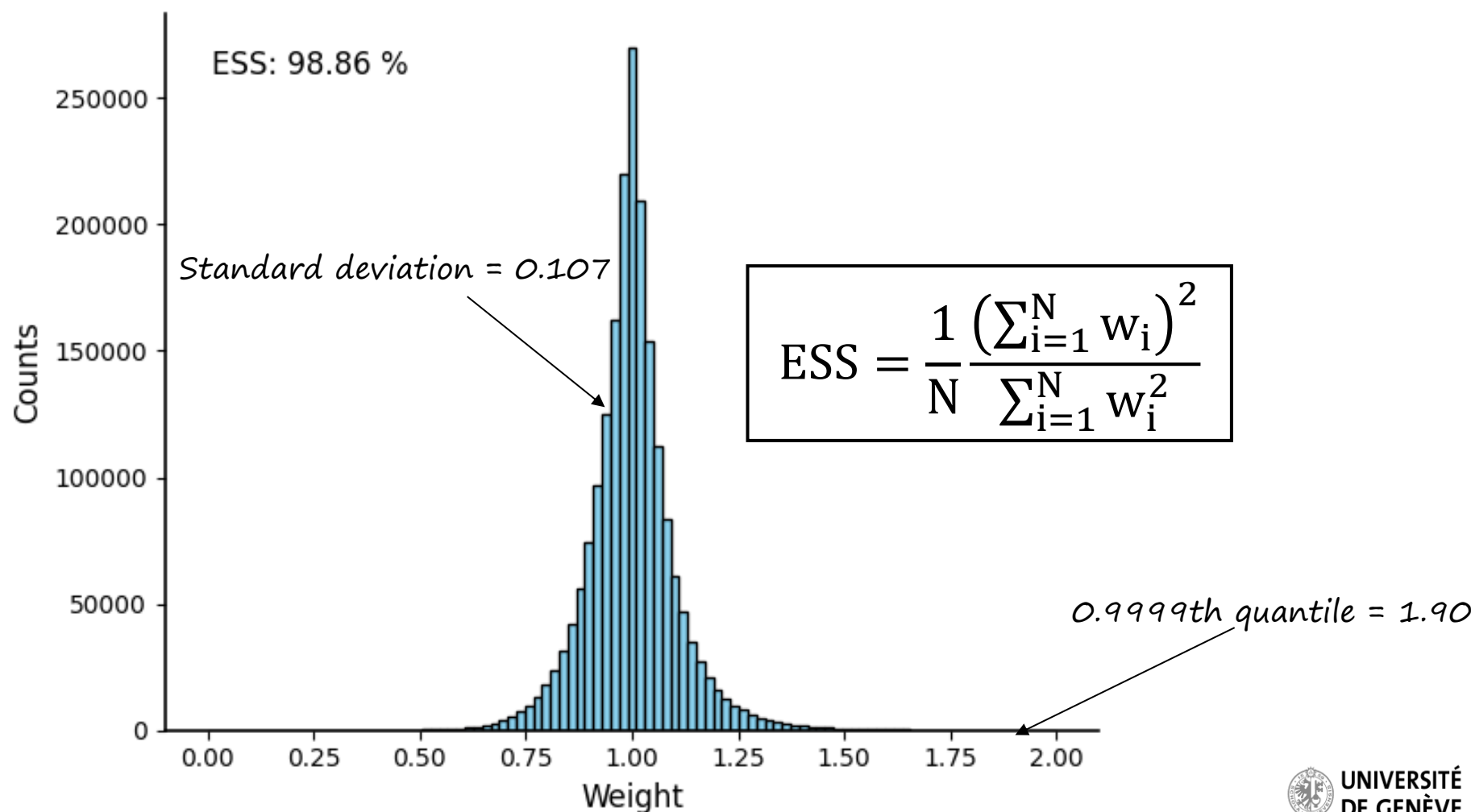
Improving unweighting efficiency

1 million events, both shells, flat flux $E_\nu = [200, 1000]$ MeV

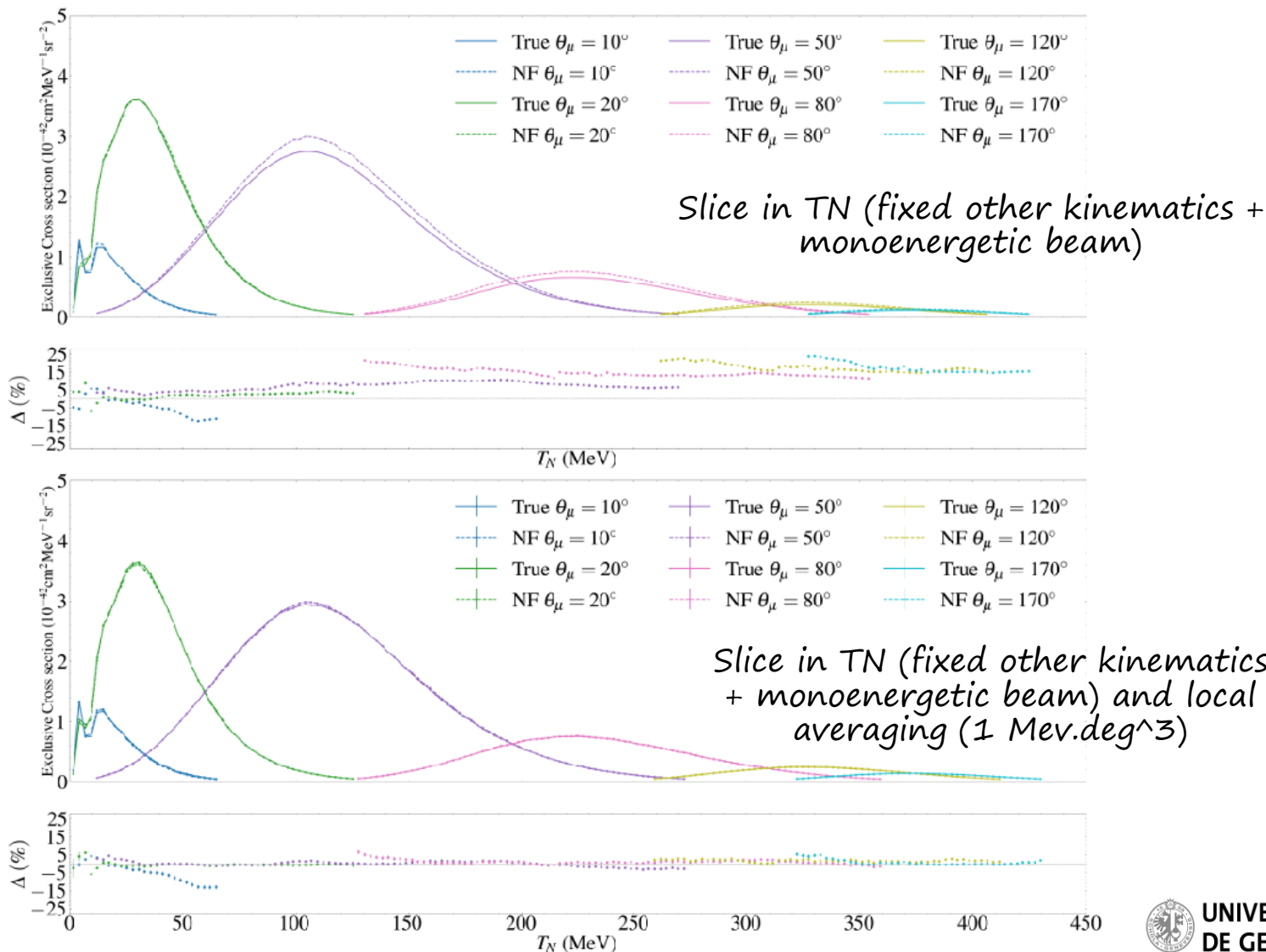


Improving unweighting efficiency

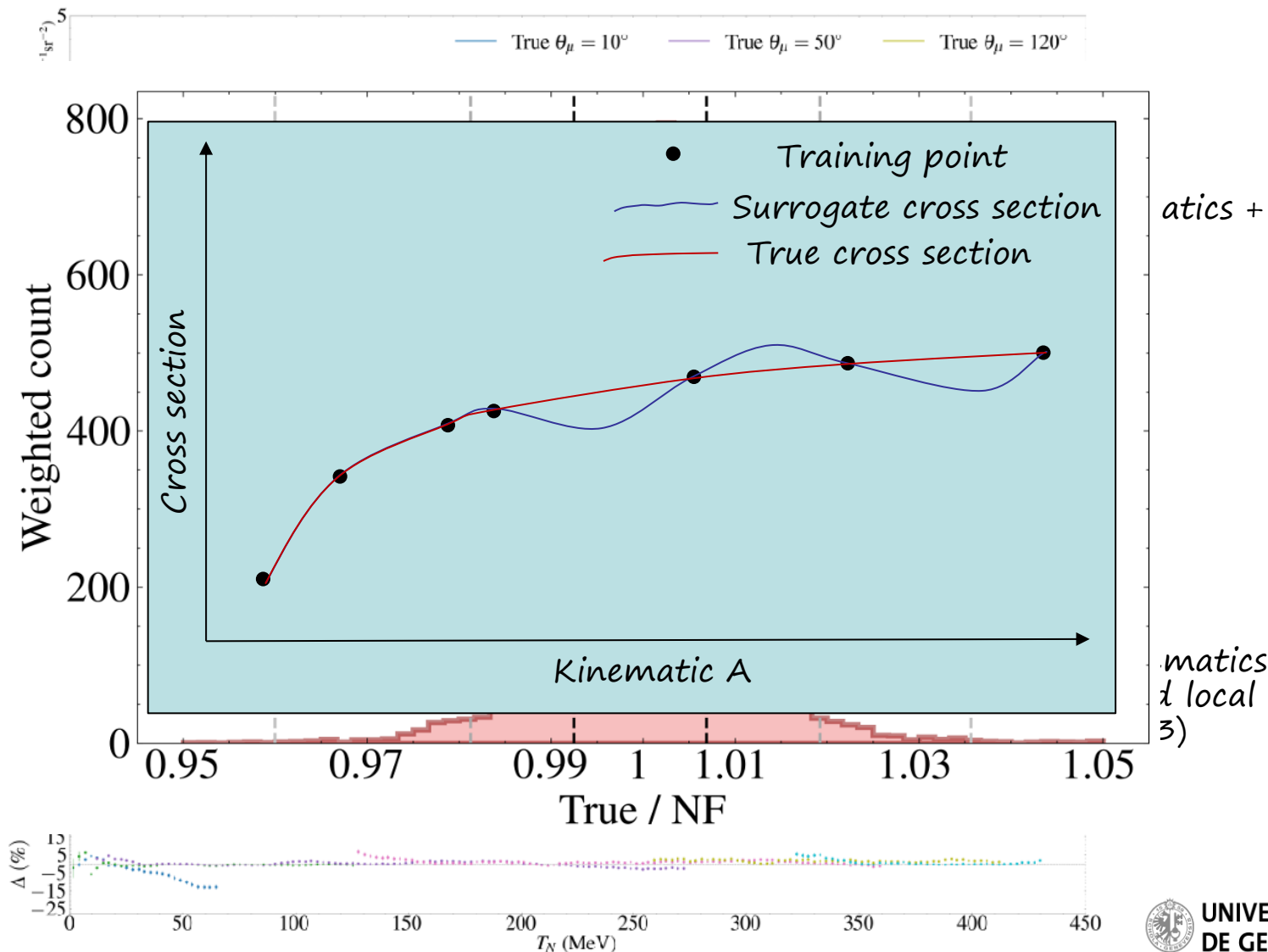
2 million events, both shells, flat flux $E_\nu = [200, 1000]$ MeV



The origin of the spread of weights



The origin of the spread of weights



Comparison with unbiased dataset

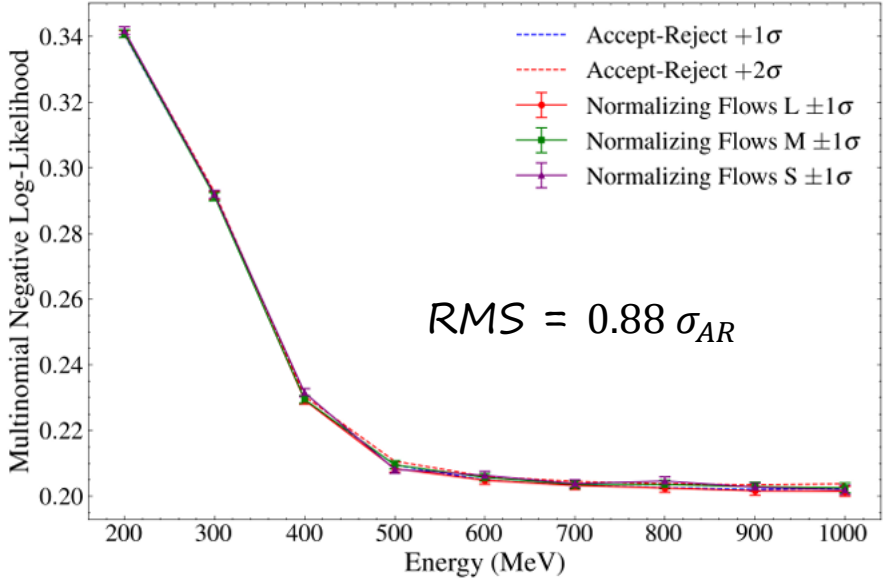
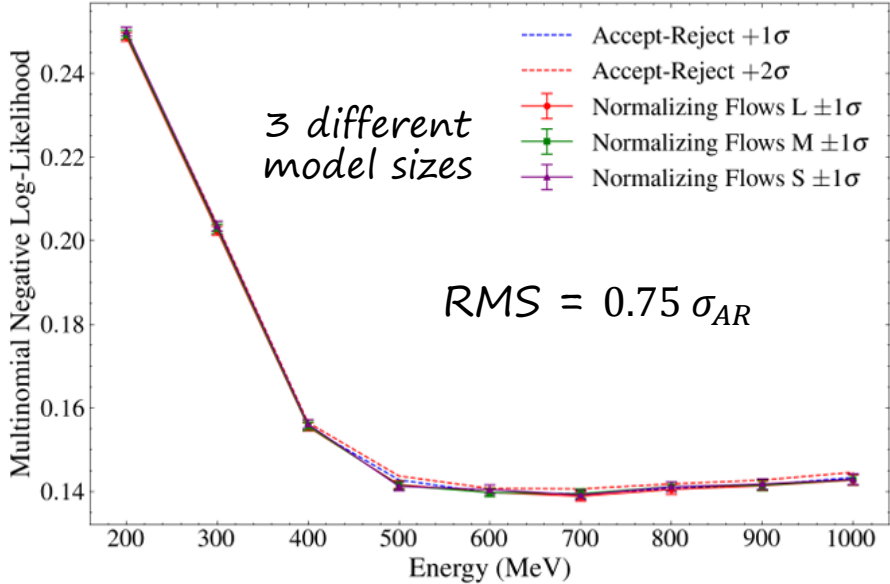
- Weight dist. : Point-by-point comparison may be too conservative
- **Binning smooths out local fluctuations.**
- Compare 4D histograms for different energy, shell combinations
 1. Histograms H_{NF}^{MC} from NF datasets (100k samples)
 2. Histograms H_{AR}^{MC} from Accept-reject datasets (100k samples)
 3. High-stat reference histogram H^{ref} (~500 unweighted samples/bin)

$$\text{MNLL}(H^{MC}) = \frac{1}{N} \sum_{\text{bin}} \left[\log(H_{\text{bin}}^{MC}!) - H_{\text{bin}}^{MC} \log\left(\frac{H_{\text{bin}}^{\text{ref}}}{N}\right) \right]$$

Multinomial Negative log-likelihood between H^{ref} and H^{MC} (NF or AR)

The MNLL for Accept-Reject datasets is the **optimal performance benchmark**.

Comparison with unbiased dataset



Evolution of the MNLL with the neutrino energy for 1s shell (left) and 1p shell (right)

Almost exclusive xsec

- We start from the **semi-exclusive** xsec where nucleon A is integrated out
- A lot of analytical simplifications and optimizations, **much faster**

5 Degrees of freedom :

$$\omega, q, \underbrace{T_A, \theta_A, \phi_A}_{\text{Integrated out}}, T_B, \theta_B, \phi_B$$

- We break the integral per shell/isospin pairs
=> **T_A is exactly known.**
- We still need to predict θ_A, ϕ_A but given already $\omega, q, T_A, T_B, \theta_B, \phi_B$
=> Not done yet, but we can produce accurate predictions by running a few exclusive points
- Most of the training points are generated for O16 and C12
=> **Ready for training**